

Preface

This course is designed for systems engineers who want to understand how LLMs work.

As a system engineer, I always wonder how things work internally and how to optimize them. I had a hard time figuring out the LLM stuff. Most of the open source projects that serve LLMs are highly optimized with CUDA kernels and other low-level optimizations. It is not easy to understand the whole picture by looking at a codebase of 100k lines of code. Therefore, I decided to implement an LLM serving project from scratch – with only matrix manipulations APIs, so that I can understand what it takes to load those LLM model parameters and do the math magic to generate text.

You can think of this course as an LLM version of CMU Deep Learning Systems course's [needle](#) project.

Prerequisites

You should have some experience with the basics of deep learning and have some idea of how PyTorch works. Some recommended resources are:

- CMU [Intro to Machine Learning](#) – this course teaches you the basics of machine learning
- CMU [Deep Learning Systems](#) – this course teaches you how to build PyTorch from scratch

Environment Setup

This course uses [MLX](#), an array/machine learning library for Apple Silicon. Nowadays it's much easier to get an Apple Silicon device than NVIDIA GPUs. In theory you can also do this course with PyTorch or numpy, but we just don't have the test infra to support them. We test your implementation against PyTorch's CPU implementation and MLX's implementation to ensure correctness.

Course Structure

This course is divided into 3 weeks. We will serve Qwen3 MLX models and optimize the serving path throughout the course.

- Week 1: serve Qwen3 with purely matrix manipulation APIs. Just Python.

- Week 2: optimizations, implement C++/Metal custom kernels to make the model run faster.
- Week 3: more optimizations, batch the requests to serve the model with high throughput.

How to Use This Book

The thing you are reading right now is the tiny-llm book. It is designed more like a guidebook instead of a textbook that explains everything from scratch. In this course, we provide the materials that we find useful on the Internet when the author(s) implemented the tiny-llm project. The Internet does a better job of explaining the concepts and I do not think it is necessary to repeat everything here. Think of this as a guide (of a list of tasks) and some hints! We will also unify the language of the Internet materials so that it is easier to correspond them to the codebase. For example, we will have a unified dimension symbols for the tensors. You do not need to figure out what H , L , E stands for and what dimension of the matrixes are passed into the function.

About the Authors

This course is created by [Chi](#) and [Connor](#).

Chi is a systems software engineer at [Neon](#) (now acquired by Databricks), focusing on storage systems. Fascinated by the vibe of large language models (LLMs), he created this course to explore how LLM inference works.

Connor is a software engineer at [PingCAP](#), developing the TiKV distributed key-value database. Curious about the internals of LLMs, he joined this course to practice how to build a high-performance LLM serving system from scratch, and contributed to building the course for the community.

Community

You may join skyzh's Discord server and study with the tiny-llm community.



Get Started

Now, you can start to set up the environment following the instructions in [Setting Up the Environment](#) and begin your journey to build tiny-llm!

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Setting Up the Environment

To follow along this course, you will need a Macintosh device with Apple Silicon. We manage the codebase with pdm.

Install pdm

Please follow the [official guide](#) to install pdm.

Clone the Repository

```
git clone https://github.com/skyzh/tiny-llm
```

The repository is organized as follows:

```
src/tiny_llm -- your implementation
src/tiny_llm_week1_ref -- reference implementation of week 1
tests/ -- unit tests for your implementation
tests_ref_impl_week1/ -- unit tests for the reference implementation of week 1
book/ -- the book
```

We provide all reference implementations and you can refer to them if you get stuck in the course.

Install Dependencies

```
cd tiny-llm
pdm install -v # this will automatically create a virtual environment and
install all dependencies
```

Check the Installation

```
pdm run check-installation
# The reference solution should pass all the *week 1* tests
pdm run test-refsol -- -- -k week_1
```

Run Unit Tests

Your code is in `src/tiny_llm`. You can run the unit tests with:

```
pdm run test
```

Download the Model Parameters

We will use the official Qwen3 MLX 4-bit model files for this course. The default model is `Qwen/Qwen3-0.6B-MLX-4bit`, which is small enough for the Week 1 dequantized Python implementation. If you have more memory, you can also try the larger Qwen3 MLX models.

Follow the guide of [this page](#) to install the Hugging Face CLI (`hf`).

The model parameters are hosted on Hugging Face. Once you authenticated your cli with the credentials, you can download them with:

```
hf auth login
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
```

Then, you can run:

```
pdm run main --solution ref --loader week1
```

It should load the model and print some text.

In week 2, we will write some kernels in C++/Metal, and we will need to set up additional tools for that. We will cover it later.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1: From Matmul to Text

In this week, we will start from the basic matrix operations and see how those these matrix manipulations can turn the Qwen3 model parameters into a model that generates text. We will implement the neural network layers used in the Qwen3 model using mlx's matrix APIs.

We will use `Qwen/Qwen3-0.6B-MLX-4bit` for this week. Week 1 dequantizes model parameters into bfloat16, so start with the 0.6B model before trying larger Qwen3 models.

What We will Cover

- Attention, Multi-Head Attention, and Grouped/Multi Query Attention
- Positional Embeddings and RoPE
- Use `mx.fast.rms_norm` for Qwen3's per-head Q/K normalization in attention, then implement RMSNorm ourselves
- Implement the MLP layer, put the attention layers together, and implement the whole Transformer model
- Load the Qwen3 model parameters and generate text

What We will Not Cover

To make the journey as interesting as possible, we will skip a few things for now:

- How to quantize/dequantize a model – that will be part of week 2. The Qwen3 model is quantized so we will need to dequantize them before we can use them in our layer implementations.
- Actually we still used some APIs other than matrix manipulations – like softmax, exp, log, etc. But they are simple and not implementing them would not affect the learning experience.
- Tokenizer – we will not implement the tokenizer from scratch. We will use the `mlx_lm` tokenizer to tokenize the input.
- Loading the model weights – I don't think it's an interesting thing to learn how to decode those tensor dump files, so we will use the `mlx_lm` to load the model and steal the weights from the loaded model into our layer implementations.

Basic Matrix APIs

Although MLX does not offer an introductory guide for beginners, its Python API is designed to be highly compatible with NumPy. To get started, you can refer to [NumPy: The Absolute Basic for Beginners](#) to learn essential matrix operations.

You can also refer to the [MLX Operations API](#) for more details.

Qwen3 Models

You can try the Qwen3 model with MLX/vLLM. You can read the blog post below to have some idea of what we will build within this course. At the end of this week, we will be able to chat with the model – that is to say, use Qwen3 to generate text, as a causal language model.

The reference implementation of the Qwen3 model can be found in huggingface transformers, vLLM, and mlx-lm. You may utilize these resources to better understand the internals of the model and what we will implement in this week.

Readings

- [Qwen3: Think Deeper, Act Faster](#)
- [Huggingface Transformers - Qwen3](#)
- [vLLM Qwen3](#)
- [mlx-lm Qwen3](#)
- [Qwen3 Technical Report](#)

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 1: Attention and Multi-Head Attention

In day 1, we will implement the basic attention layer and the multi-head attention layer. Attention layers take a input sequence and focus on different parts of the sequence when generating the output. Attention layers are the key building blocks of the Transformer models.

 [Reading: Transformer Architecture](#)

We use the Qwen3 model for text generation. The model is a decoder-only model. The input of the model is a sequence of token embeddings. The output of the model is the most likely next token ID.

 [Reading: LLM Inference, the Decode Phase](#)

Back to the attention layer. The attention layer takes a query, a key, and a value. In a classic implementation, all of them are of the same shape: $N \times L \times D$.

N is zero or some number of dimensions for batches. Within each of the batch, L is the sequence length and D is the dimension of the embedding for a given head in the sequence.

So, for example, if we have a sequence of 1024 tokens, where each of the token has a 512-dimensional embedding (head_dim), we will pass a tensor of the shape $N \times 1024 \times 512$ to the attention layer.

Task 1: Implement scaled_dot_product_attention_simple

In this task, we will implement the scaled dot product attention function. We assume the input tensors (Q, K, V) have the same dimensions. In the next few chapters, we will support more variants of attentions that might not have the same dimensions for all tensors.

```
src/tiny_llm/attention.py
```

Readings

- [Annotated Transformer](#)
- [PyTorch Scaled Dot Product Attention API](#) (assume `enable_gqa=False`, assume `dim_k=dim_v=dim_q` and `H_k=H_v=H_q`)
- [MLX Scaled Dot Product Attention API](#) (assume `dim_k=dim_v=dim_q` and `H_k=H_v=H_q`)

- **Attention is All You Need**

Implement `scaled_dot_product_attention` following the below attention function. The function takes key, value, and query of the same dimensions, and an optional mask matrix `M`.

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Note that $\frac{1}{\sqrt{d_k}}$ is the scale factor. The user might specify their own scale factor or use the default one.

`L` is `seq_len`, in PyTorch API it's `S` (source len)
`D` is `head_dim`

key: `N.. x L x D`
 value: `N.. x L x D`
 query: `N.. x L x D`
 output: `N.. x L x D`
 scale = `1/sqrt(D)` if not specified

You may use `softmax` provided by `mlx` and implement it later in week 2.

Because we are always using the attention layer within the multi-head attention layer, the actual tensor shape when serving the model will be:

key: `1 x H x L x D`
 value: `1 x H x L x D`
 query: `1 x H x L x D`
 output: `1 x H x L x D`
 mask: `1 x H x L x L`

.. though the attention layer only cares about the last two dimensions. The test case will test any shape of the batching dimension.

At the end of this task, you should be able to pass the following tests:

```
pdm run test --week 1 --day 1 -- -k task_1
```

Task 2: Implement SimpleMultiHeadAttention

In this task, we will implement the multi-head attention layer.

`src/tiny_llm/attention.py`

Readings

- [Annotated Transformer](#)
- [PyTorch MultiHeadAttention API](#) (assume $\text{dim}_k=\text{dim}_v=\text{dim}_q$ and $H_k=H_v=H_q$)
- [MLX MultiHeadAttention API](#) (assume $\text{dim}_k=\text{dim}_v=\text{dim}_q$ and $H_k=H_v=H_q$)
- [The Illustrated GPT-2 \(Visualizing Transformer Language Models\)](#) helps you better understand what key, value, and query are.

Implement `SimpleMultiHeadAttention`. The layer takes a batch of vectors, maps it through the K, V, Q weight matrixes, and use the attention function we implemented in task 1 to compute the result. The output needs to be mapped using the O weight matrix.

You will also need to implement the `linear` function in `basics.py` first. For `linear`, it takes a tensor of the shape $N \times I$, a weight matrix of the shape $O \times I$, and a bias vector of the shape O . The output is of the shape $N \times O$. I is the input dimension and O is the output dimension.

For the `SimpleMultiHeadAttention` layer, the input tensors `query`, `key`, `value` have the shape $N \times L \times E$, where E is the dimension of the embedding for a given token in the sequence. The `K/Q/V` weight matrixes will map the tensor into key, value, and query separately, where the dimension E will be mapped into a dimension of size $H \times D$, which means that the token embedding gets mapped into H heads, each with a dimension of D . You can directly reshape the tensor to split the $H \times D$ dimension into two dimensions of H and D to get H heads for the token.

Now, you have a tensor of the shape $N \times L \times H \times D$ for each of the key, value, and query. To apply the attention function, you first need to transpose them into shape $N \times H \times L \times D$.

- This makes each attention head an independent batch, so that attention can be calculated separately for each head across the sequence L .
- If you kept H behind L , attention calculation would mix head and sequence dimensions, which is not what we want — each head should focus only on the relationships between tokens in its own subspace.

The attention function produces output for each of the head of the token. Then, you can transpose it back into $N \times L \times H \times D$ and reshape it so that all heads get merged back together with a shape of $N \times L \times (H \times D)$. Map it through the output weight matrix to get the final output.

E is `hidden_size` or `embed_dim` or `dims` or `model_dim`
 H is `num_heads`
 D is `head_dim`
 L is `seq_len`, in PyTorch API it's `S` (source len)

$w_q/w_k/w_v$: $(H \times D) \times E$
 output/input: $N \times L \times E$
 w_o : $E \times (H \times D)$

At the end of the task, you should be able to pass the following tests:

```
pdm run test --week 1 --day 1 -- -k task_2
```

You can run all tests for the day with:

```
pdm run test --week 1 --day 1
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 2: Positional Encodings and RoPE

In day 2, we will implement the positional embedding used in the Qwen3 model: Rotary Positional Encoding. In a transformer model, we need a way to embed the information of the position of a token into the input of the attention layers. In Qwen3, positional embedding is applied within the multi head attention layer on the query and key vectors.

Readings

- [You could have designed state of the art positional encoding](#)
- [Roformer: Enhanced Transformer with Rotary Positional Encoding](#)

Task 1: Implement Rotary Positional Encoding “RoPE”

You will need to modify the following file:

```
src/tiny_llm/positional_encoding.py
```

In traditional RoPE (as described in the readings), the positional encoding is applied to each head of the query and key vectors. You can pre-compute the frequencies when initializing the `RoPE` class.

If `offset` is not provided, the positional encoding will be applied to the entire sequence: 0th frequency applied to the 0th token, up to the (L-1)-th token. Otherwise, the positional encoding will be applied to the sequence according to the offset slice. If the offset slice is 5..10, then the sequence length provided to the layer would be 5, and the 0th token will be applied with the 5th frequency.

You *only* need to consider `offset` being `None` or a single slice. The `list[slice]` case will be implemented when we start implementing the continuous batching feature. Assume all batches provided use the same offset.

```
x: (N, L, H, D)
cos/sin_freqs: (MAX_SEQ_LEN, D // 2)
```

In the traditional form of RoPE, each head on the dimension of `D` is viewed as consecutive complex pairs. That is to say, if `D = 8`, then, `x[0]` and `x[1]` are a pair, `x[2]` and `x[3]` are another pair, and so on. A pair gets the same frequency from `cos/sin_freqs`.

Note that, practically, `D` can be even or odd. In the case of `D` being odd, the last dimension of `x` doesn't have a matching pair, and is typically left untouched in most implementations.

For simplicity, we just assume that D is always even.

```
output[0] = x[0] * cos_freqs[0] + x[1] * -sin_freqs[0]
output[1] = x[0] * sin_freqs[0] + x[1] * cos_freqs[0]
output[2] = x[2] * cos_freqs[1] + x[3] * -sin_freqs[1]
output[3] = x[2] * sin_freqs[1] + x[3] * cos_freqs[1]
...and so on
```

You can do this by reshaping x to $(N, L, H, D // 2, 2)$ and then applying the above formula to each pair.

Readings

- [PyTorch RotaryPositionalEmbeddings API](#)
- [MLX Implementation of RoPE before the custom metal kernel implementation](#)

You can test your implementation by running the following command:

```
pdm run test --week 1 --day 2 -- -k task_1
```

Task 2: Implement RoPE in the non-traditional form

The Qwen3 model uses a non-traditional form of RoPE. In this form, the head embedding dimension is split into two halves, and the two halves are applied with different frequencies. Let's say $x_1 = x[., :HALF_DIM]$ and $x_2 = x[., HALF_DIM:]$.

```
output[0] = x1[0] * cos_freqs[0] + x2[0] * -sin_freqs[0]
output[HALF_DIM] = x1[0] * sin_freqs[0] + x2[0] * cos_freqs[0]
output[1] = x1[1] * cos_freqs[1] + x2[1] * -sin_freqs[1]
output[HALF_DIM + 1] = x1[1] * sin_freqs[1] + x2[1] * cos_freqs[1]
...and so on
```

You can do this by directly getting the first half / second half of the embedding dimension of x and applying the frequencies to each half separately.

Readings

- [vLLM implementation of RoPE](#)

You can test your implementation by running the following command:

```
pdm run test --week 1 --day 2 -- -k task_2
```

At the end of the day, you should be able to pass all tests of this day:

```
pdm run test --week 1 --day 2
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 3: Grouped Query Attention (GQA)

In day 3, we will implement Grouped Query Attention (GQA). The Qwen3 models use GQA which is an optimization technique for multi-head attention that reduces the computational and memory costs associated with the Key (K) and Value (V) projections. Instead of each Query (Q) head having its own K and V heads (like in Multi-Head Attention, MHA), multiple Q heads share the same K and V heads. Multi-Query Attention (MQA) is a special case of GQA where all Q heads share a single K/V head pair.

Readings

- [GQA Paper](#)
- [Qwen layers implementation in mlx-lm](#)
- [PyTorch API \(the case where enable_gqa=True\)](#)
- [torch.nn.MultiHeadAttention](#)

Task 1: Implement `scaled_dot_product_attention_grouped`

You will need to modify the following file:

```
src/tiny_llm/attention.py
```

In this task, we will implement the grouped scaled dot product attention function, which forms the core of GQA.

Implement `scaled_dot_product_attention_grouped` in `src/tiny_llm/attention.py`. This function is similar to the standard scaled dot product attention, but handles the case where the number of query heads is a multiple of the number of key/value heads.

The main progress is the same as the standard scaled dot product attention. The difference is that the K and V heads are shared across multiple Q heads. This means that instead of having H_q separate K and V heads, we have H K and V heads, and each K and V head is shared by $n_repeats = H_q // H$ Q heads.

The core idea is to reshape `query`, `key`, and `value` so that the K and V tensors can be effectively broadcasted to match the query heads within their groups during the `matmul` operations. * Think about how to isolate the `H` and `n_repeats` dimensions in the `query` tensor. * Consider adding a dimension of size 1 for `n_repeats` in the `key` and `value` tensors to enable broadcasting. Then perform the scaled dot product attention calculation

(`matmul`, `scale`, optional `mask`, `softmax`, `matmul`). Broadcasting should handle the head repetition implicitly.

Note that, leverage broadcasting instead of repeating the K and V tensors is more efficient. This is because broadcasting allows the same data to be used in multiple places without creating multiple copies of the data, which can save memory and improve performance.

At last, don't forget to reshape the final result back to the expected output shape.

`N..` is zero or more dimensions for batches
`H_q` is the number of query heads
`H` is the number of key/value heads (`H_q` must be divisible by `H`)
`L` is the query sequence length
`S` is the key/value sequence length
`D` is the head dimension

query: `N.. x H_q x L x D`
key: `N.. x H x S x D`
value: `N.. x H x S x D`
mask: `N.. x H_q x L x S`
output: `N.. x H_q x L x D`

Please note that besides the grouped heads, we also extend the implementation that Q, K, and V might not have the same sequence length.

You can test your implementation by running the following command:

```
pdm run test --week 1 --day 3 -- -k task_1
```

Task 2: Causal Masking

Readings

- [Writing an LLM from scratch, part 9 – causal attention](#)

In this task, we will implement the causal masking for the grouped attention.

The causal masking is a technique that prevents the attention mechanism from attending to future tokens in the sequence. When `mask` is set to `causal`, we will apply the causal mask.

The causal mask is a square matrix of shape (L, S) , where L is the query sequence length and S is the key/value sequence length. The mask is a lower triangular matrix, where the elements on the diagonal and below the diagonal are 0, and the elements above the diagonal are $-\infty$. For example, if $L = 3$ and $S = 5$, the mask will be:


```
0  0  0  -inf -inf
0  0  0  0    -inf
0  0  0  0     0
```

Please implement the `causal_mask` function in `src/tiny_llm/attention.py` and then use it in the `scaled_dot_product_attention_grouped` function. Also note that our causal mask diagonal position is different from the PyTorch API.

You can test your implementation by running the following command:

```
pdm run test --week 1 --day 3 -- -k task_2
```

Task 3: Qwen3 Grouped Query Attention

In this task, we will implement the Qwen3 Grouped Query Attention. You will need to modify the following file:

```
src/tiny_llm/qwen3_week1.py
```

`Qwen3MultiHeadAttention` implements the multi-head attention for Qwen3. You will need to implement the following pseudo code:

```
x: B, L, E
q = linear(x, wq) -> B, L, H_q, D
k = linear(x, wk) -> B, L, H, D
v = linear(x, wv) -> B, L, H, D
q = rms_norm(q, q_norm)
k = rms_norm(k, k_norm)
q = rope(q, offset=slice(0, L))
k = rope(k, offset=slice(0, L))
(transpose as needed)
x = scaled_dot_product_attention_grouped(q, k, v, scale, mask) -> B, L, H_q, D
; Do this at float32 precision
(transpose as needed)
x = linear(x, wo) -> B, L, E
```

Qwen3 attention has no Q/K/V projection bias, and it applies RMSNorm to each Q/K head before RoPE. We will implement the general `RMSNorm` layer on day 4, so for today call `mx.fast.rms_norm` directly for `q_norm` and `k_norm`. Keep in mind that you should use non-traditional RoPE.

You can test your implementation by running the following command:

```
pdm run test --week 1 --day 3 -- -k task_3
```

At the end of the day, you should be able to pass all tests of this day:

```
pdm run test --week 1 --day 3
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 4: RMSNorm and Multi Perceptron Layer

In day 4, we will implement two crucial components of the Qwen3 Transformer architecture: RMSNorm and the MLP (Multi-Layer Perceptron) block, also known as the FeedForward Network. RMSNorm is a layer normalization technique that helps stabilize training with less computational overhead compared to traditional layer normalization. The MLP block is a feedforward network that processes the output of the attention layers, applying non-linear transformations to enhance the model's expressiveness.

Task 1: Implement RMSNorm

In this task, we will implement the `RMSNorm` layer.

```
src/tiny_llm/layer_norm.py
```

Day 3 used `mx.fast.rms_norm` directly so that the GQA chapter could stay focused on attention. This task implements the same normalization rule as a reusable layer. After this point, the transformer block, final model norm, and any Q/K normalization path can use your own `RMSNorm` implementation instead of treating normalization as a built-in API.

Readings

- [Root Mean Square Layer Normalization](#)
- [Qwen3 layers implementation in mlx-lm \(includes RMSNorm\)](#) - See `RMSNorm`.

RMSNorm is defined as:

$$y = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \cdot \text{weight}$$

Where:

- `x` is the input tensor.
- `weight` is a learnable scaling parameter.
- `epsilon` (eps) is a small constant added for numerical stability (e.g., 1e-5 or 1e-6).
- `mean(x^2)` is the sum of squares and then division by the number of elements.

The normalization is applied independently to each sample's feature vector, typically over the last dimension of input. Note that, mean calculation should be performed with `float32` accumulation to maintain precision before taking the square root, even if the input and weights are in a lower precision format (e.g., `float16` or `bfloat16`). After computing the

normalized value, cast it back to the original input dtype before applying `weight`. This matches the low-precision path used by MLX's fast RMSNorm kernels: the normalization statistics are accumulated in `float32`, while the final scaling by `weight` happens in the model dtype.

`D` is the embedding dimension.

```
x: N.. x D
weight: D
output: N.. x D
```

You can test your implementation by running:

```
pdm run test --week 1 --day 4 -- -k task_1
```

Task 2: Implement the MLP Block

In this task, we will implement the MLP block named `Qwen3MLP`.

```
src/tiny_llm/qwen3_week1.py
```

The original Transformer model utilized a simple Feed-Forward Network (FFN) within each block. This FFN typically consisted of two linear transformations with a ReLU activation in between, applied position-wise.

Modern Transformer architectures, including Qwen3, often employ more advanced FFN variants for improved performance. Qwen3 uses a specific type of Gated Linear Unit (GLU) called SwiGLU.

A plain FFN can be abstracted as:

```
h = activation(W_up(x))
out = W_down(h)
```

GLU keeps the same expand-then-project-back shape, but adds another projection that gates the intermediate features before `W_down`. This gives the MLP a learned, input-dependent way to control which intermediate channels matter, instead of only applying an activation to the same features produced by `W_up`.

SwiGLU is the GLU variant used by Qwen3:

```
u = W_up(x)
g = SiLU(W_gate(x))
out = W_down(g * u)
```

Readings

- [Attention is All You Need \(Transformer Paper, Section 3.3 “Position-wise Feed-Forward Networks”\)](#)
- [GLU Paper\(Language Modeling with Gated Convolutional Networks\)](#)
- [SiLU\(Swish\) activation function](#)
- [SwiGLU Paper\(GLU Variants Improve Transformer\)](#)
- [PyTorch SiLU documentation](#)
- [Qwen3 layers implementation in mlx-lm \(includes MLP\)](#)

Essentially, SwiGLU is a combination of GLU and the SiLU (Sigmoid Linear Unit) activation function:

- GLU is a gating mechanism that allows the model to learn which parts of the input to focus on. It typically involves an element-wise product of two linear projections of the input, one of which might be passed through an activation function. Compared to ReLU used in the original FFN, GLU can help the model learn more complex relationships in the data, deciding which features to keep and which to discard.
- SiLU (Sigmoid Linear Unit) is a smooth, non-monotonic activation function that has been shown to perform well in various deep learning tasks. Compared to ReLU and sigmoid used in GLU, it is fully differentiable without the zero-gradient “dead zones”, retains non-zero output even for negative inputs.

You need to implement the `silu` function in `basics.py` first. For `silu`, it takes a tensor of the shape `N.. x I` and returns a tensor of the same shape. The `silu` function is defined as:

$$\text{SiLU}(x) = x * \text{sigmoid}(x) = \frac{x}{1 + e^{-x}}$$

Compute the sigmoid part in a numerically stable way:

```
if x >= 0:
    sigmoid(x) = 1 / (1 + exp(-x))
else:
    sigmoid(x) = exp(x) / (1 + exp(x))
```

The negative branch is algebraically equivalent to the direct sigmoid formula, but it avoids `exp(-x)` becoming `exp(large positive)` when `x` is a large negative value. In vector code, this can be expressed with `abs(x)`: compute the direct branch using `|x|`, then use `1 - y` for negative inputs. That matches MLX’s low-precision GPU path more closely than the direct division form.

Then implement `Qwen3MLP`. The structure for Qwen3’s MLP block is:

- A gate linear projection (W_{gate}).
- An up linear projection (W_{up}).
- A SiLU activation function applied to the output of W_{gate} .

- An element-wise multiplication of the SiLU-activated W_{gate} output and the W_{up} output. This forms the “gated” part.
- A final down linear projection (W_{down}).

This can be expressed as:

$$\text{MLP}(x) = (\text{SiLU}(W_{gate}(x)) \odot W_{up}(x))W_{down}$$

Where $\odot$ denotes element-wise multiplication. All linear projections in Qwen3’s MLP are typically implemented without bias.

N.. is zero or more dimensions for batches
 E is hidden_size (embedding dimension of the model)
 I is intermediate_size (dimension of the hidden layer in MLP)
 L is the sequence length

input: N.. x L x E
 w_gate: I x E
 w_up: I x E
 w_down: E x I
 output: N.. x L x E

You can test your implementation by running:

```
pdm run test --week 1 --day 4 -- -k task_2
```

At the end of the day, you should be able to pass all tests of this day:

```
pdm run test --week 1 --day 4
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 5: The Qwen3 Model

In day 5, we will implement the Qwen3 model.

Before we start, please make sure you have downloaded the models:

```
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
```

Otherwise, some of the tests will be skipped.

Task 1: Implement Qwen3TransformerBlock

```
src/tiny_llm/qwen3_week1.py
```

Readings

- [A Simplified Explanation of the Transformer Block](#)
- [Attention is All You Need](#)

Qwen3 uses the following transformer block structure:

```
input
/ |
| input_layernorm (RMSNorm)
| |
| Qwen3MultiHeadAttention
\ |
  Add (residual)
/ |
| post_attention_layernorm (RMSNorm)
| |
| MLP
\ |
  Add (residual)
|
output
```

You should pass all tests for this task by running:

```
# Download the models if you haven't done so
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
# Run the tests
pdm run test --week 1 --day 5 -- -k task_1
```

Task 2: Implement Embedding

src/tiny_llm/embedding.py

Readings

- [LLM Embeddings Explained: A Visual and Intuitive Guide](#)

The embedding layer maps one or more tokens (represented as an integer) to one or more vector of dimension `embedding_dim`. In this task, you will implement the embedding layer.

```
Embedding::__call__
weight: vocab_size x embedding_dim
Input: N.. (tokens)
Output: N.. x embedding_dim (vectors)
```

This can be done with a simple array index lookup operation.

In the Qwen3 model, the embedding layer can also be used as a linear layer to map the embeddings back to the token space.

```
Embedding::as_linear
weight: vocab_size x embedding_dim
Input: N.. x embedding_dim
Output: N.. x vocab_size
```

You should pass all tests for this task by running:

```
# Download the models if you haven't done so; we need to tokenizers
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
# Run the tests
pdm run test --week 1 --day 5 -- -k task_2
```


Task 3: Implement Qwen3ModelWeek1

Now that we have built all the components of the Qwen3 model, we can implement the `Qwen3ModelWeek1` class.

```
src/tiny_llm/qwen3_week1.py
```

Readings

In this course, you will not implement the process of loading the model parameters from the tensor files. Instead, we will load the model using the `mlx-lm` library, and then we will place the loaded parameters into our model. Therefore, the `Qwen3ModelWeek1` class will take a MLX model as the constructor argument.

The Qwen3 model has the following layers:

```
input
| (tokens: N..)
Embedding
| (N.. x hidden_size); note that hidden_size==embedding_dim
Qwen3TransformerBlock
| (N.. x hidden_size)
Qwen3TransformerBlock
| (N.. x hidden_size)
...
|
RMSNorm
| (N.. x hidden_size)
Embedding::as_linear OR Linear (lm_head)
| (N.. x vocab_size)
output
```

You can access the number of layers, hidden size, head dimension, and other model parameters from `mlx_model.args` which is defined in [ModelArgs](#). You can reach the loaded weights from `mlx_model.model`; the layer names are easiest to inspect from the Qwen3 MLX model metadata on Hugging Face.

By this point, you have implemented `RMSNorm` yourself. If your day 3 attention path still calls `mx.fast.rms_norm` for `q_norm` and `k_norm`, you can now replace those calls with `RMSNorm(head_dim, q_norm, eps=...)` and `RMSNorm(head_dim, k_norm, eps=...)`. They implement the same formula; the built-in call existed only to avoid teaching `RMSNorm` before the GQA chapter.

Note that different size of the Qwen3 models use different strategies to map the embeddings back to the token space. Some models directly use the `Embedding::as_linear` layer, while others have a separate `lm_head` linear layer. You can decide which strategy to use based on the `mlx_model.args.tie_word_embeddings` argument. If it is true, then you

should use `Embedding::as_linear`. Otherwise, the `lm_head` linear layer will be available and you should load its parameters.

The input to the model is a sequence of tokens. The output is the logits (probability distribution) of the next token. In the next day, we will implement the process of generating the response from the model, and decide the next token based on the probability distribution output.

Also note that the MLX model we are using is a quantized model. Therefore, you also need to dequantize the weights before loading them into our tiny-llm model. You can use the provided `quantize::dequantize_linear` function to dequantize the weights.

You also need to make sure that you set `mask=causal` when the input sequence is longer than 1. We will explain why in the next day.

You should pass all tests for this task by running:

```
# Download the models if you haven't done so
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
# Run the tests
pdm run test --week 1 --day 5 -- -k task_3
```

At the end of the day, you should be able to pass all tests of this day:

```
pdm run test --week 1 --day 5
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 6: Generating the Response: Prefill and Decode

In day 6, we will implement the process of generating the response when using the LLM as a chatbot. The implementation is not a lot of code, but given that it uses a large portion of the code we implemented in the previous days, we want to allocate this day to debug the implementation and make sure everything is working as expected.

Task 1: Implement `simple_generate`

```
src/tiny_llm/generate.py
```

The `simple_generate` function takes a model, a tokenizer, and a prompt, and generates the response. The generation process is done in two parts: first prefill, and then decode.

First thing is to implement the `_step` sub-function. It takes a list of tokens `y`. The model will return the logits: the probability distribution of the next token for each position.

```
y: N.. x S, where in week 1 we don't implement batch, so N.. = 1
output_logits: N.. x S x vocab_size
```

You only need the last token's logits to decide the next token. Therefore, you need to select the last token's logits from the output logits.

```
logits = output_logits[:, -1, :]
```

Then, you can optionally apply the log-sum-exp trick to normalize the logits to avoid numerical instability. As we only do argmax sampling, the log-sum-exp trick is not necessary. Then, you need to sample the next token from the logits. You can use the `mx.argmax` function to sample the token with the highest probability over the last dimension (the `vocab_size` axis). The function returns the next token number. This decoding strategy is called greedy decoding as we always pick the token with the highest probability.

-  [The Log-Sum-Exp Trick](#)
-  [Decoding Strategies in Large Language Models](#)
-  [Tokenizer definition](#)

With the `_step` function implemented, you can now implement the full `simple_generate` function. The function will first prefill the model with the prompt. As the prompt is a string, you need to first convert it to a list of tokens by using the tokenizer `tokenizer.encode`.

You will need to implement a while loop to keep generating the response until the model outputs the EOS `tokenizer.eos_token_id` token. In the loop, you will need to store all previous tokens in a list, and use the detokenizer `tokenizer.detokenizer` to print the response.

An example of the sequences provided to the `_step` function is as below:

```
tokenized_prompt: [1, 2, 3, 4, 5, 6]
prefill: _step(model, [1, 2, 3, 4, 5, 6]) # returns 7
decode: _step(model, [1, 2, 3, 4, 5, 6, 7]) # returns 8
decode: _step(model, [1, 2, 3, 4, 5, 6, 7, 8]) # returns 9
...
```

We will optimize the `decode` process to use key-value cache to speed up the generation next week.

You can test your implementation by running the following command:

```
# Download the models if you haven't done so
hf download Qwen/Qwen3-0.6B-MLX-4bit
hf download Qwen/Qwen3-1.7B-MLX-4bit
hf download Qwen/Qwen3-4B-MLX-4bit
# Run the tests
pdm run main --solution tiny_llm --loader week1 --model qwen3-0.6b \
  --prompt "Give me a short introduction to large language model"
pdm run main --solution tiny_llm --loader week1 --model qwen3-1.7b \
  --prompt "Give me a short introduction to large language model"
pdm run main --solution tiny_llm --loader week1 --model qwen3-4b \
  --prompt "Give me a short introduction to large language model"
```

It should gives you a reasonable response of “what is a large language model”. Replace `--solution tiny_llm` with `--solution ref` to use the reference solution.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 1 Day 7: Sampling and Preparing for Week 2

In day 7, we will implement various sampling strategies. And we will get you prepared for week 2.

Task 1: Sampling

We implemented the default greedy sampling strategy in the previous day. In this task, we will implement the temperature, top-k, and top-p (nucleus) sampling strategies.

```
src/tiny_llm/sampler.py
```

-  [mlx-lm sampler implementation](#)

Temperature Sampling

The first sampling strategy is the temperature sampling. When `temp=0`, we use the default greedy strategy. When it is larger than 0, we will randomly select the next token based on the logprobs. The temperature parameter scales the distribution. When the value is larger, the distribution will be more uniform, making the lower probability token more likely to be selected, and therefore making the model more creative.

To implement temperature sampling, simply divide the logprobs by the temperature and use `mx.random.categorical` to randomly select the next token.

```
pdm run main --solution tiny_llm --loader week1 --model qwen3-0.6b --sampler-temp 0.5
```

Top-k Sampling

In top-k sampling, we will only keep the top-k tokens with the highest probabilities before sampling the probabilities. This is done before the final temperature scaling.

You can use `mx.argmaxpartition` to partition the output so that you can know the indices of the top-k elements, and then, mask those logprobs outside the top-k with `-mx.inf`. After that, do temperature sampling.

```
pdm run main --solution tiny_llm --loader week1 --model qwen3-0.6b --sampler-temp 0.5 --sampler-top-k 10
```

Top-p (Nucleus) Sampling

In top-p (nucleus) sampling, we will only keep the top-p tokens with the highest cumulative probabilities before sampling the probabilities. This is done before the final temperature scaling.

There are multiple ways of implementing it. One way is to first use `mx.argsort` to sort the logprobs (from highest probability to lowest), and then, do a `cumsum` over the sorted logprobs to get the cumulative probabilities. Then, mask those logprobs outside the top-p with `-mx.inf`. After that, do temperature sampling.

```
pdm run main --solution tiny_llm --loader week1 --model qwen3-0.6b --sampler-temp 0.5 --sampler-top-p 0.9
```

Task 2: Prepare for Week 2

In week 2, we will optimize the serving infrastructure of the Qwen3 model. We will write some C++ code and Metal kernel to make some operations run faster. You will need Xcode and its command-line tools, which include the Metal compiler, to compile the C++ code and Metal kernels.

1. **Install Xcode:** Install Xcode from the Mac App Store or from the [Apple Developer website](#) (this may require an Apple Developer account).
2. **Launch Xcode and Install Components:** After installation, launch Xcode at least once. It may prompt you to install additional macOS components; please do so (this is usually the default option).
3. **Install Xcode Command Line Tools:** Open your Terminal and run:

```
xcode-select --install
```

4. **Set Default Xcode Path (if needed):** Ensure that your command-line tools are pointing to your newly installed Xcode. You can do this by running:

```
sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
```

(Adjust the path if your Xcode is installed in a different location).

5. **Accept Xcode License:** You may also need to accept the Xcode license:

```
sudo xcodebuild -license accept
```

6. **Install CMake:**

```
brew install cmake
```

(This instruction is graciously provided by [Liu Jinyi](#).)

You can test your installation by compiling the code in `src/extensions` with a `axpby` function as part of the official mlx extension tutorial:

```
pdm run build-ext  
pdm run build-ext-test
```

It should print `correct: True`.

If you are not familiar with C++ or Metal programming, we also suggest doing some small exercises to get familiar with them. You can implement some element-wise operations like `exp`, `sin`, `cos` and replace the MLX ones in your model implementation.

That's all for week 1! We have implemented all the components to serve the Qwen3 model. Now we are ready to start week 2, where we will optimize the serving infrastructure and make it run blazing fast on your Apple Silicon device.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 2: Tiny vLLM

In Week 2 of the course, we will focus on building serving infrastructure for the Qwen3 model. Essentially, this means creating a minimal version of the vLLM project from scratch. By the end of the week, you'll be able to serve the Qwen3 model efficiently on your Apple Silicon device using the infrastructure we've built together.

What We'll Cover

- Key-value cache implementation
- C++/Metal kernels
 - Implementing a quantized matmul kernel
 - Implementing a flash attention kernel
 - Note: This week, we won't focus on performance optimization. The kernels you build will likely be around 10x slower than MLX implementations. Optimizing them will be left as an exercise.
- Model serving infrastructure
 - Implementing chunked prefill
 - Implementing continuous batching

This week continues with Qwen3 as the main model. The serving code uses the official Qwen3 MLX 4-bit model files, preserves their bfloat16 tensors, and builds the KV cache, custom kernels, and batching path around that model family.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 2 Day 1: Key-Value Cache

In this chapter, we will implement the **key-value cache** for the Qwen3 model. The key-value cache is an essential component of the attention mechanism, as it allows the model to reuse previously computed results instead of recomputing them for every new token.

Readings

- [KV Caching Explained: Optimizing Transformer Inference Efficiency](#)

Recall from last week how we supplied data to the model:

```
tokenized_prompt: [1, 2, 3, 4, 5, 6]
prefill: _step(model, [1, 2, 3, 4, 5, 6]) # returns 7
decode: _step(model, [1, 2, 3, 4, 5, 6, 7]) # returns 8
decode: _step(model, [1, 2, 3, 4, 5, 6, 7, 8]) # returns 9
...
```

```
x: B, L, E
q = linear(x, wq) -> B, L, H_q, D
k = linear(x, wk) -> B, L, H, D
v = linear(x, wv) -> B, L, H, D
q = rms_norm(q, q_norm)
k = rms_norm(k, k_norm)
q = rope(q, offset=slice(offset, offset + L))
k = rope(k, offset=slice(offset, offset + L))
(transpose as needed)
x = scaled_dot_product_attention_grouped(q, k, v, scale, mask) -> B, L, H_q, D
# at float32 precision
(transpose as needed)
x = linear(x, wo) -> B, L, E
```

The attention mechanism is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

Consider two consecutive decoding steps with $L = S = 3$ and $L = S = 4$, where each head in each layer has an embedding dim of $D = 4$:

```

L = 3
Q      x  K^T      =
1 1 1 1    1 2 3    1x1  -inf -inf
2 2 2 2    1 2 3    2x1  2x2  -inf
3 3 3 3    1 2 3    3x1  3x2  3x3
           1 2 3

```

```

L = 4
Q      x  K^T      =
1 1 1 1    1 2 3 4    1x1  -inf -inf -inf
2 2 2 2    1 2 3 4    2x1  2x2  -inf -inf
3 3 3 3    1 2 3 4    3x1  3x2  3x3  -inf
4 4 4 4    1 2 3 4    4x1  4x2  4x3  4x4

```

Notice that the first three rows/cols of $Q \times K^T$ are identical in both steps. Also given that we are using the causal masks, we do not need to care about the upper triangle of the matrix. The same applies to the softmax function and the multiplication with the V matrix. This means we are unnecessarily recomputing results for tokens we've already processed, and the new information only comes from the last row of $Q \times K^T$.

The solution is to cache the K and V matrices and only compute new values for incoming tokens:

K in cache:

```

1 1 1 1
2 2 2 2

```

[a b c d] represent cached values

```

L = 1, S = 3
Q      x  K^T      =
           (⬇ is K not transposed)
           [1 1 1 1]
           [2 2 2 2]
3 3 3 3    3 3 3 3    3x1  3x2  3x3

```

```

L = 1, S = 4
Q      x  K^T      =
           (⬇ is K not transposed)
           [1 1 1 1]
           [2 2 2 2]
           [3 3 3 3]
4 4 4 4    4 4 4 4    4x1  4x2  4x3  4x4

```

Task 1: Implement the Key-Value Cache

src/tiny_llm/kv_cache.py

Each layer in the model maintains its own key-value cache. The cache has a single API, `update_and_fetch`, which:

1. Takes the newly computed κ and v for incoming tokens.
2. Concatenates them with the existing cached matrices.
3. Returns the full cached κ and v .

For week 2 day 1, you only need to handle `key` and `value`. The `mask` and `mask_length` parameters will remain unused.

You may implement this in `kv_cache.py` as `TinyKvFullCache`:

```
L' = new tokens length
L  = total tokens length

update_and_fetch(key, value) -> key, value

key:   B, L', H, D
value: B, L', H, D

self.key   = concat_or_initialize(self.key, key, on the L' dimension)
self.value = concat_or_initialize(self.value, value, on the L' dimension)

self.key:   B, L, H, D
self.value: B, L, H, D

return self.key, self.value
```

Task 2: Use the Key-Value Cache

`src/tiny_llm/qwen3_week2.py`

With the cache in place, update your week 1 Qwen3 implementation to support it. Implement the `Qwen3MultiHeadAttention` class in `qwen3_week2.py`.

- Each layer should use its own cache.
- The model must now accept an `offset` argument, which represents the position of the last token processed.
- This value should match the current sequence length in the cache (you can add assertions to check consistency).
- Both the argument and the cache maintain the offset for debugging purposes.

Example computation flow:

```

x: B, L', E
q = linear(x, wq) -> B, L', H_q, D
k = linear(x, wk) -> B, L', H, D
v = linear(x, wv) -> B, L', H, D
q = rms_norm(q, q_norm)
k = rms_norm(k, k_norm)
q = rope(q, offset=slice(offset, offset + L'))
k = rope(k, offset=slice(offset, offset + L'))
(transpose as needed)
k, v = cache.update_and_fetch(k, v) ; k/v: B, L, H, D, q: B, L', H, D
x = scaled_dot_product_attention_grouped(q, k, v, scale, mask) -> B, L', H_q, D
# at float32 precision
(transpose as needed)
x = linear(x, wo) -> B, L', E

```

We use two different variables for the L' because they have different meanings in the context of this chapter and the context of week 1 day 3: in the GQA implementation, k/v 's sequence length is s (source length), while q 's sequence length is L . In the Qwen3 multihead attention implementation, L' is the “new token” and L is the total sequence length, which corresponds to L and s in week 1 respectively.

Note that another refactor of this week's code is that all modules now take `QuantizedWeights` instead of `mx.array` for some weights. You will need to move the dequantize code from loading the model to each module first, and we will replace it with our own quantized matmul implementation for the rest of the week.

Task 3: Implement the Model

`src/tiny_llm/qwen3_week2.py`

Complete the rest of the model using your week 1 implementation as a base, but modify all relevant components to use the key-value cache.

To verify correctness, run the following test (almost identical to week 1's test):

```
pdm run test --week 2 --day 1
```

Task 4: Implement Decoding

`src/tiny_llm/generate.py`

Next, implement the decoding logic in `generate.py` by completing the `simple_generate_with_kv_cache` function. This function should call your Week 2 Qwen3

model with both the `offset` and the newly decoded token.

For example:

```
tokenized_prompt: [1, 2, 3, 4, 5, 6]
prefill: _step(model, [1, 2, 3, 4, 5, 6], 0) # returns 7
decode:  _step(model, [7], 7) # returns 8
decode:  _step(model, [8], 8) # returns 9
...
```

You can test your implementation with:

```
pdm run main --solution tiny_llm --loader week2 --model qwen3-0.6b
pdm run main --solution tiny_llm --loader week2 --model qwen3-4b
```

You can also benchmark throughput and compare your implementation with the reference solution:

```
pdm bench --solution tiny_llm --loader week2 --model qwen3-0.6b
pdm bench --solution tiny_llm_ref --loader week2 --model qwen3-0.6b
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 2 Day 2-3: Quantized Matmul

In this chapter, we will implement the quantized matrix multiplication. Quantization compresses model weights from 16-bit floating point to 4-bit integers, which is critical for efficient LLM serving on devices with limited memory bandwidth.

Readings

- [Model Compression and Quantization](#)
- [MLX Extensions Development Guide](#)
- [Quantized Matmul on CPU \(Video\)](#)
- [Quantized Matmul on GPU \(Video\)](#)

Why Quantization?

As we learned in the KV Cache chapter, the decode phase of LLM inference is **memory-bandwidth bound**. Let's revisit the arithmetic intensity calculation for the Qwen3-0.6B model:

Per-token linear layers in decode phase:

- Input: 1 token $\times$ 1024 dimensions = 1024 bfloat16 values = 2 KB
- MLP weights: $1024 \times 3072 \times 3$ matrices $\times$ 2 bytes = ~19 MB per layer
- Attention weights:
 - q_proj / o_proj: $1024 \times 2048 \times 2$ matrices $\times$ 2 bytes = ~8 MB per layer
 - k_proj / v_proj: $1024 \times 1024 \times 2$ matrices $\times$ 2 bytes = ~4 MB per layer
- Total weights per layer: ~31 MB
- Total for 28 layers: ~880 MB

FLOPs (2 per multiply-accumulate):

- MLP per layer: $2 \times 3 \times 1024 \times 3072 \approx 19\text{M}$
- Attention projections per layer: $2 \times (1024 \times 2048 \times 2 + 1024 \times 1024 \times 2) \approx 13\text{M}$
- 28 layers: ~880 million per token

Memory access: ~880 MB

Arithmetic intensity: $880\text{M FLOPs} / 880 \text{ MB} \approx 1.0 \text{ FLOPs/Byte}$

With M3 Max's 400 GB/s memory bandwidth and ~10 TFLOPS compute:

Memory-bound throughput: $400 \text{ GB/s} \times 1.0 \text{ FLOPs/Byte} = 400 \text{ GFLOPS}$

Compute-bound throughput: 10 TFLOPS

We're using only ~4% of available compute!

The Solution: Quantization

By compressing weights from 16-bit floating point (`float16` or `bfloat16`) to 4-bit integers (`int4`), we:

- **Reduce memory bandwidth by 4×**: 880 MB $\rightarrow$ ~220 MB per token
- **Improve arithmetic intensity by 4×**: 1.0 $\rightarrow$ ~4.0 FLOPs/Byte
- **Increase throughput by ~4×**: 400 GFLOPS $\rightarrow$ ~1.6 TFLOPS

The tradeoff is minimal accuracy loss with proper quantization techniques.

Group-wise Quantization

Instead of quantizing all weights uniformly, we divide them into **groups** and quantize each group independently. This preserves more information about the weight distribution.

For a weight matrix W of shape (K, N) , we divide each row into groups of size G . In this course we use Qwen3 MLX 4-bit weights, whose group size is fixed at 128:

Original weight matrix W : $K \times N$ (`float16` or `bfloat16`)

Group size: $G = 128$

Number of groups per row = N / G

For each group of G consecutive values in a row:

1. Find min and max values
2. Compute scale and bias to map $[\text{min}, \text{max}] \rightarrow [0, 15]$ (4-bit range)
3. Quantize each value using: $\text{quantized} = \text{round}((\text{value} - \text{bias}) / \text{scale})$

All quantized matmul tests use `group_size = 128`, matching the Qwen3 MLX 4-bit weights used by the rest of the course. The tests cover both `float16` and `bfloat16` because different MLX checkpoints store their scales, biases, and activations in different 16-bit data types.

Affine Quantization

We use **affine (asymmetric) quantization** which maps a floating-point range to the full integer range:

$$\text{quantized} = \text{round} \left(\frac{\text{value} - \text{bias}}{\text{scale}} \right)$$

$$\text{dequantized} = \text{quantized} \times \text{scale} + \text{bias}$$

For 4-bit quantization, the quantized values are in the range $[0, 15]$.

Given a group with minimum value $v_{\min}$ and maximum value $v_{\max}$:

$$\text{scale} = \frac{v_{\max} - v_{\min}}{2^{\text{bits}} - 1} = \frac{v_{\max} - v_{\min}}{15}$$

$$\text{bias} = v_{\min}$$

Example:

Group values: [-0.5, -0.3, 0.1, 0.4, 0.8]
 min = -0.5, max = 0.8

scale = (0.8 - (-0.5)) / 15 = 1.3 / 15 ≈ 0.0867
 bias = -0.5

Quantization:

-0.5 → round((-0.5 - (-0.5)) / 0.0867) = 0
 -0.3 → round((-0.3 - (-0.5)) / 0.0867) = 2
 0.1 → round((0.1 - (-0.5)) / 0.0867) = 7
 0.4 → round((0.4 - (-0.5)) / 0.0867) = 10
 0.8 → round((0.8 - (-0.5)) / 0.0867) = 15

Quantized: [0, 2, 7, 10, 15] (4 bits each)

Storage Format

For efficient storage and computation, quantized weights are packed:

Original: $K \times N$ float16/bfloat16 (2 bytes each) = $2KN$ bytes
 Quantized: $K \times N$ int4 (0.5 bytes each) = $0.5KN$ bytes

Packing: 8×4 -bit values fit in one uint32 (32 bits)

Weight matrix shape: $K \times N$

Quantized storage shape: $K \times (N / 8)$ uint32

Scales shape: $K \times (N / G)$ float16/bfloat16

Biases shape: $K \times (N / G)$ float16/bfloat16

Example packing for 8 consecutive 4-bit values [a, b, c, d, e, f, g, h]:

```
uint32_value = (h << 28) | (g << 24) | (f << 20) | (e << 16) |
               (d << 12) | (c << 8) | (b << 4) | a
```

Unpacking:

```
a = (uint32_value >> 0) & 0xF
b = (uint32_value >> 4) & 0xF
c = (uint32_value >> 8) & 0xF
...
h = (uint32_value >> 28) & 0xF
```


Quantized Matrix Multiplication

Mathematical Formulation

For standard matrix multiplication $C = AB^T$ where:

- A : shape (M, N) , float16 or bfloat16 (activations)
- B : shape (K, N) , **quantized** to int4 (weights)
- C : shape (M, K) , same 16-bit dtype as A (output)

Each element $C[i, k]$ is computed as:

$$C[i, k] = \sum_{j=0}^{N-1} A[i, j] \times B[k, j]$$

With quantization, $B[k, j]$ is represented as:

$$B[k, j] = B_{\text{quantized}}[k, j] \times \text{scale}[k, g] + \text{bias}[k, g]$$

where $g = \lfloor j/G \rfloor$ is the group index.

Substituting:

$$C[i, k] = \sum_{g=0}^{N/G-1} \sum_{j'=0}^{G-1} A[i, g \times G + j'] \times (B_{\text{quantized}}[k, g \times G + j'] \times \text{scale}[k, g] + \text{bias}[k, g])$$

Rearranging:

$$C[i, k] = \sum_{g=0}^{N/G-1} \left(\text{scale}[k, g] \sum_{j'=0}^{G-1} A[i, g \times G + j'] \times B_{\text{quantized}}[k, g \times G + j'] + \text{bias}[k, g] \right)$$

This shows we can factor out the scale and bias per group, reducing the number of floating-point operations.

Computation Flow

Input:

A: $M \times N$ (float16 or bfloat16, activations)
B_quantized: $K \times (N/8)$ (uint32, packed weights)
scales: $K \times (N/G)$ (float16/bfloat16)
biases: $K \times (N/G)$ (float16/bfloat16)

Output:

C: $M \times K$ (float16/bfloat16)

For each output element C[i, k]:

```
sum = 0 # float accumulator
for each group g in 0..(N/G - 1):
    scale = scales[k, g]
    bias = biases[k, g]

    # Process G values in the group (G/8 uint32 packs)
    for each pack p in 0..(G/8 - 1):
        packed_value = B_quantized[k, g*(G/8) + p]

        # Unpack 8 × 4-bit values
        for bit_offset in [0, 4, 8, 12, 16, 20, 24, 28]:
            quantized = (packed_value >> bit_offset) & 0xF
            b_value = quantized * scale + bias
            a_value = A[i, g*G + p*8 + bit_offset/4]
            sum = sum + a_value * b_value
```

C[i, k] = float16/bfloat16(sum)

Task 1: Implement QuantizedWeights and QuantizedEmbedding

src/tiny_llm/quantize.py
src/tiny_llm/embedding.py

First, familiarize yourself with the `QuantizedWeights` class, which stores quantized weight information:

Field	Shape	Description
weight	$(K, N/8)$ uint32	Packed quantized weights. Each uint32 stores 8 consecutive 4-bit values. The original weight matrix has shape (K, N) , and after packing, it becomes $(K, N/8)$.
scales	$(K, N/G)$ float16/bfloat16	Per-group scale factors for dequantization. Each group of G consecutive values shares

Field	Shape	Description
		one scale. Recall: $\text{scale} = (v_{max} - v_{min})/15$
biases	$(K, N/G)$ float16/bfloat16	Per-group bias (offset) for dequantization. Recall: $\text{bias} = v_{min}$
group_size	int	Number of consecutive values that share the same scale/bias. For the Qwen3 MLX 4-bit weights used here, this is 128.
bits	int	Quantization bit width (typically 4, meaning values are in range $[0, 15]$)

The `from_mlx_layer` static method extracts these fields from MLX's quantized linear layers when loading the model.

Next, implement the `quantized_linear` function, which is a wrapper around `quantized_matmul` that mimics the standard `linear` function interface. And we'll implement `quantized_matmul` in the next task.

The token embedding table should also stay quantized in Week 2. Add a `QuantizedEmbedding` wrapper with two call patterns:

- `embedding(input_ids)` is a row lookup. Gather the matching packed rows, scales, and biases, then call `mx.dequantize` on only those selected rows.
- `embedding.as_linear(h)` is the tied output projection. Implement this with `quantized_linear(h, embedding_weight)` so it uses your quantized matmul path instead of materializing the full `vocab_size x hidden_size` table. This path starts working once the quantized matmul kernel is implemented in the next tasks.

Task 2: Implement `quantized_matmul` (CPU version)

In this task, we will implement the quantized matmul as an MLX C++ extension. The pattern is identical to the existing `axpby` example in the codebase — read through `axpby.h`, `axpby.cpp`, and the corresponding binding in `bindings.cpp` first as your reference.

```
src/extensions/src/tiny_llm_ext.h
src/extensions/bindings.cpp
src/extensions/src/quantized_matmul.cpp
src/extensions/CMakeLists.txt
```

You need to touch three files, all within the `tiny_llm_ext` namespace:

- **`tiny_llm_ext.h`** — Declare the `quantized_matmul(...)` function signature and define a `QuantizedMatmul` primitive class (inheriting `mx::Primitive`). Store `group_size` and

`bits` as private members.

- **bindings.cpp** — Add an `m.def(...)` call to expose the function to Python.
- **quantized_matmul.cpp** — Implement the `quantized_matmul(...)` function (validate inputs, compute output shape, return a lazy `mx::array`) and the `eval_cpu` method (allocate output, register arrays with the CPU encoder, dispatch the compute kernel).

The `eval_cpu` implementation follows the same CPU encoder pattern as `axpby`: allocate output memory with `out.set_data(mx::allocator::malloc(out.nbytes()))`, register input/output arrays with the encoder, then dispatch a lambda that performs the actual computation. Inside the lambda, implement the nested loop from the Computation Flow section above — iterate over each output element `(i, k)`, dequantize each packed value, accumulate the products in `float`, and write the result back as either `float16` or `bfloat16`, matching the input dtype.

Follow the `axpby` dtype-dispatch pattern here: write the CPU implementation as a template, then dispatch with `mx::float16_t` or `mx::bfloat16_t` based on the output dtype.

Don't forget to add `src/quantized_matmul.cpp` to `target_sources` in `CMakeLists.txt`.

You can test your implementation by running:

```
pdm run build-ext
pdm run test --week 2 --day 2 -- -k task_2
```

Task 3: Implement quantized_matmul (GPU version)

```
src/extensions/src/quantized_matmul.metal
src/extensions/src/quantized_matmul.cpp
```

In this task, you will write the Metal kernel for quantized matmul **and** wire up the `eval_gpu` method to dispatch it. Keep the math exactly the same as Task 2 (CPU); only the execution model changes.

Metal Kernel

You need to implement one kernel entry in `quantized_matmul.metal`:

- Use a **one-thread-per-output-element** mapping: each thread computes `out[i, k]`.
- The kernel should support both `half` and `bfloat16_t` inputs and outputs.
- Apply the same group-wise dequantization loop as the CPU version:
 - Iterate over groups of 128 values
 - Unpack int4 values from packed `uint32`
 - Dequantize with `q * scale + bias`

- Accumulate the products in `float` and cast the final output back to the kernel dtype
- Add boundary checks (`i < M`, `k < K`) before writing output.

The custom kernel only needs to handle `bits = 4` and `group_size = 128`. Use that group size to compute `groups_per_row` and the packed weight offsets. Instantiate the same templated Metal kernel twice, once for `half` and once for `bfloat16_t`, and select the matching kernel name in `eval_gpu`.

GPU Dispatch

Complete the `eval_gpu` method in `quantized_matmul.cpp` to dispatch your Metal kernel. Follow the same pattern as `axpby`'s GPU dispatch:

1. Get the Metal device and command encoder from the stream.
2. Load the quantized matmul kernel matching the output dtype from the Metal library.
3. Set input/output buffers and dimension constants (`M`, `N`, `K`) on the encoder — make sure the buffer order matches your kernel signature.
4. Calculate a 2D thread group configuration: use `kernel->maxTotalThreadsPerThreadgroup()` to determine the total threads, then split between the `M` and `K` dimensions (e.g., 32 threads for `M`, the rest for `K`).
5. Dispatch with `dispatchThreadgroups`.

You can test your implementation by running:

```
pdm run build-ext
pdm run test --week 2 --day 2 -- -k task_3
```

Task 4: Model Integration

```
src/tiny_llm/qwen3_week2.py
```

Integrate your quantized matmul into the Week 2 Qwen3 model so that inference runs on quantized weights end-to-end.

Change the weight type from `mx.array` to `QuantizedWeights` for all linear layers in attention (`wq/wk/wv/wo`) and MLP (`w_gate/w_up/w_down`). Replace every `linear(x, w)` call with `quantized_linear(x, w)`. In the model loading code, use `QuantizedWeights.from_mlx_layer(...)` to extract quantized weight information from each MLX linear layer, instead of calling `mx.dequantize` to get a full 16-bit matrix. Make sure the Week 1 loader still dequantizes (since Week 1 layers expect plain `mx.array`), while the Week 2 loader does **not** dequantize.

For embeddings, wire the `QuantizedEmbedding` from Task 1 into the loader: load `embed_tokens` with `QuantizedWeights.from_mlx_layer(...)` and pass it to `QuantizedEmbedding`. If the model has a separate `lm_head`, keep that head as `QuantizedWeights` too and apply it with `quantized_linear`; `lm_head` is a projection, not an embedding lookup.

Qwen3 MLX quantized layers may use **float16** or **bfloat16** for the tensors involved in dequantization. Your kernel should accept `scales`, `biases`, and activations in either dtype, require them to match, and return the same dtype. If you see `nan` or garbage output, a dtype mismatch is the most likely cause.

Also keep the quantized layer's parameters. The model code should pass through `w.group_size` and `w.bits`; the extension should validate that they match the Qwen3 course assumptions: `group_size = 128` and `bits = 4`.

You can test your implementation by running:

```
pdm run main --solution tiny_llm --loader week2 --model qwen3-0.6b
```

You can also benchmark throughput and compare your implementation with the reference solution:

```
pdm bench --solution tiny_llm --loader week2 --model qwen3-0.6b
pdm bench --solution tiny_llm_ref --loader week2 --model qwen3-0.6b
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 2 Day 4-5: Flash Attention 2

In this chapter, we will implement Flash Attention 2 for the Week 2 Qwen3 serving pipeline. The goal is to replace the regular attention path with a tiled implementation to reduce memory bandwidth and increase throughput, especially for long contexts.

Readings

- [From Online Softmax to FlashAttention](#)
- [FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)
- [FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning](#)
- [MLX Extension Development Guide](#)
- [MLX steel attention kernel \(reference\)](#)

Why Flash Attention?

The key idea from the FlashAttention papers is that attention is often **IO-bound**, not FLOP-bound.

In the standard implementation, we compute:

1. $S = QK^T$
2. $P = \text{softmax}(S + \text{mask})$
3. $O = PV$

This path materializes large $L \times S$ tensors (S and often P) in global memory. For long contexts, repeatedly writing and reading these tensors dominates runtime.

For example, if $L = S = 4096$:

One $L \times S$ matrix: $4096 \times 4096 = 16,777,216$ elements
 float32 storage: ~64 MB per matrix per head
 Scores + probabilities: ~128 MB temporary memory per head

So even before counting $Q/K/V$ and output tensors, memory traffic is already huge.

IO-Aware Exact Attention

FlashAttention avoids this bottleneck by tiling $Q/K/V$ into on-chip memory (cache / shared memory), and combining each tile with **online softmax** updates. Instead of storing the full attention matrix, it keeps only per-row running statistics (m , τ) and partial output (o).

This gives three practical benefits:

- **Exactness:** same result as standard softmax attention (not an approximation).
- **Lower memory:** activation memory scales linearly with sequence length instead of quadratically.
- **Higher throughput:** fewer high-bandwidth-memory accesses, which is usually the real bottleneck.

Online Softmax Recap

For one query row, split keys/values into tiles $j = 1..T$:

$$\begin{aligned}
 m^{(j)} &= \max \left(m^{(j-1)}, \max(s^{(j)}) \right) \\
 l^{(j)} &= e^{m^{(j-1)} - m^{(j)}} l^{(j-1)} + \sum e^{s^{(j)} - m^{(j)}} \\
 o^{(j)} &= e^{m^{(j-1)} - m^{(j)}} o^{(j-1)} + \sum e^{s^{(j)} - m^{(j)}} v^{(j)}
 \end{aligned}$$

At the end:

$$o = \frac{o^{(T)}}{l^{(T)}}$$

This is the core numerical trick used by both the CPU and GPU kernels in this chapter, and the rest of the implementation is mostly about mapping this update rule to CPU loops and Metal threadgroups.

Task 1: Implement `flash_attention Wrapper`

`src/tiny_llm/attention.py`

Implement `flash_attention(query, key, value, scale=None, mask=None)` so it matches the extension API in `tiny_llm_ext`.

Follow the same shape convention as Week 1 and Week 2 attention:

```

query: B..., H_q, L, E
key:   B..., H,   S, E
value: B..., H,   S, E
mask:  B..., H_q, L, S
out:   B..., H_q, L, E

```

The wrapper should compute `factor` using `mx.rsqrt` when `scale` is `None`, flatten batch and head dimensions before calling into C++, and reshape the output back to the original

layout. Make sure `query`, `key`, and `value` are contiguous before calling the extension. For `mask`, always broadcast to `B...`, `H_q`, `L`, `s`, reshape to `(N, L, s)`, and cast to `float32` so that CPU and GPU kernels receive exactly the same dtype.

Task 2: Implement `flash_attention` (CPU version)

```
src/extensions/src/tiny_llm_ext.h
src/extensions/bindings.cpp
src/extensions/src/flash_attention.cpp
src/extensions/CMakeLists.txt
```

In this task, add the new MLX primitive and its CPU implementation. The structure is the same as the quantized matmul chapter: declare the primitive in `tiny_llm_ext.h`, expose it in `bindings.cpp`, and register `flash_attention.cpp` in `CMakeLists.txt`.

Before creating the lazy output array, validate all shape and dtype constraints in C++: inputs should be 3D float32 tensors, `num_heads` must be divisible by `num_kv_heads`, and head mapping between Q and KV batches must be consistent.

Then implement `FlashAttention::eval_cpu(...)` with tiled online softmax. Use `Br = 32` and `Bc = 32`, and the rationale for this choice will be explained in the GPU section. Iterate over `(n, i, j)` tiles, map query heads to KV heads with `q_kv_heads_ratio = num_heads / num_kv_heads`, and accumulate in float32. Mask values should be applied in each tile before updating `m_i` and `l_i`.

When `mask == "causal"`, treat it as a block-level optimization opportunity: if a tile is fully invalid, skip that tile entirely; if a tile is fully valid, skip mask read/add for that tile and continue with matmul + online softmax. Also note that `L` and `s` are not always equal in causal attention, so do not hardcode logic that assumes `L == s`.

You can test your implementation by running:

```
pdm run build-ext
pdm run test --week 2 --day 4 -- -k task_2
```

Task 3: Implement `flash_attention` (GPU version)

```
src/extensions/src/flash_attention.metal
src/extensions/src/flash_attention.cpp
src/extensions/CMakeLists.txt
```

Now implement the GPU path for the same algorithm.

GPU Parallelization Strategy

The key to an efficient GPU implementation is understanding how to map the tiled algorithm to Metal's execution model.

Why Br = 32 and Bc = 32?

The tile sizes are not arbitrary—they are constrained by Apple GPU hardware:

Constraint	Source	Value
SIMD width	Apple GPU fixed	32
Max threads per threadgroup	Hardware limit	1024
Bc	= SIMD width (for efficient <code>simd_sum / simd_max</code>)	32
Br	= 1024 / 32	32
Threadgroup memory	32KB limit	Fits <code>q_local[32][128]</code> + <code>o_i[32][128]</code>

With Br=32 and Bc=32, we get $32 \times 32 = 1024$ threads per threadgroup, which exactly fills the hardware limit.

Grid and Threadgroup Layout

Grid (num_threadgroups):

TG(0, 0) head=0, qtile=0	TG(1, 0) head=1, qtile=0	TG(2, 0) head=2, qtile=0
TG(0, 1) head=0, qtile=1	TG(1, 1) head=1, qtile=1	TG(2, 1) head=2, qtile=1
...	...	...

X: N (heads)

Y: Tr (query blocks)

Each threadgroup is responsible for one (head, Q-tile) output block.

Thread Mapping Within a Threadgroup

Each threadgroup handles one Q block (size Br×E) for one head. Inside the threadgroup:

Threadgroup = 32 SIMD groups × 32 threads/group = 1024 threads

SIMD group 0	→ Q[0, :] (handles row 0)	← 32 threads
SIMD group 1	→ Q[1, :] (handles row 1)	← 32 threads
SIMD group 2	→ Q[2, :] (handles row 2)	← 32 threads
...		
SIMD group 31	→ Q[31, :] (handles row 31)	← 32 threads

Inside that single threadgroup, the kernel runs a **serial** loop over all K/V tiles $j = 0..T_c-1$.

Computing $S = Q @ K^T$

Each thread computes one element of the 32×32 score matrix. Here's how the matrix multiplication maps to threads:

Q block [Br=32, E=128]

Q[0,:] (128 elements)
Q[1,:]
Q[2,:]
...
Q[31,:]

↑
simd_gid = a
(which row)

K^T [E=128, Bc=32]

K	K	K	...	K
[0]	[1]	[2]	...	[31]
T	T	T		T
128	128	128		128

@

↑
simd_lid = b
(which column)

Result: S block [Br=32, Bc=32], each element computed by one thread:

		simd_lid (b)					
		0	1	2	...	31	
simd_gid (a)	0	S0,0	S0,1	S0,2	...	S0,31	← SIMD group 0 (32 threads)
	1	S1,0	S1,1	S1,2	...	S1,31	← SIMD group 1
	2	S2,0	S2,1	S2,2	...	S2,31	← SIMD group 2
	...	...	...	...	...	...	
	31	S31,0	S31,1	S31,2	...	S31,31	← SIMD group 31

Thread (a=2, b=5) computes:

$S[2,5] = Q[2,0]*K[5,0] + Q[2,1]*K[5,1] + \dots + Q[2,127]*K[5,127]$
 = dot product of Q row 2 with K row 5 (128 multiply-adds)

After computing $S[a,b]$, each thread holds one attention score. Row-wise reductions use SIMD intrinsics—all 32 threads in the same SIMD group cooperate:

SIMD group 2 (threads with `simd_gid=2`):

Thread `b=0` has $S[2,0]$

Thread `b=1` has $S[2,1]$

...

Thread `b=31` has $S[2,31]$

`simd_max(s_a_b)` → all 32 threads get $\max(S[2,0], S[2,1], \dots, S[2,31])$

`simd_sum(p_a_b)` → all 32 threads get $\sum(P[2,0], P[2,1], \dots, P[2,31])$

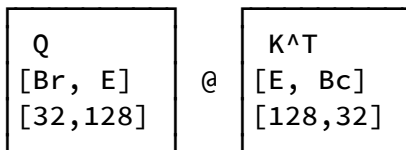
```
float rowmax = simd_max(s_a_b); // max across 32 threads in same SIMD group
```

```
float rowsum = simd_sum(p_a_b); // sum across 32 threads in same SIMD group
```

Computing $O = P @ V$ inside a SIMD group

After softmax, we need to accumulate the output tile. A natural first thought is: “Can we assign threads to output elements the same way we did for $S = Q @ K^T$?” The answer is **no**, because the output dimensions don’t match:

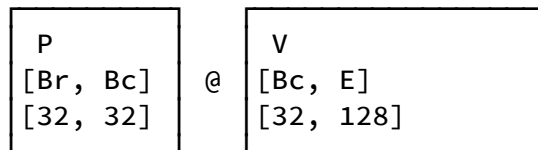
$Q @ K^T$:



↓
 $S [Br, Bc]$
 $[32, 32]$
 = 1024 elements

↓
 1024 threads ✓
 (one per element)

$P @ V$:



↓
 $O [Br, E]$
 $[32, 128]$
 = 4096 elements

↓
 1024 threads ✗
 (not enough!)

For $S = Q @ K^T$, we have 1024 output elements and 1024 threads—perfect one-to-one mapping. But for $O = P @ V$, we have 4096 output elements but only 1024 threads. The mismatch comes from the embedding dimension: **$E = 128 \neq Bc = 32$** .

So we use a different strategy: instead of assigning threads to output columns, we **loop over the 128 output columns** and use SIMD reduction for each:

For each output element $O[a, c]$:

$$O[a, c] = \underbrace{\text{sum over } b: P[a, b] * V[b, c]}_{\substack{32 \text{ terms } (Bc = 32) \\ \downarrow \\ \text{simd_sum can handle this!}}}$$

Thread assignment:

- `simd_gid` = `a` (which output row)
- `simd_lid` = `b` (which term in the sum)

Code:

```
for c in 0..E-1:                // loop 128 times
    val = P[a, b] * V[b, c]      // each lane computes one term
    result = simd_sum(val)        // reduce 32 terms → 1 result
    if simd_lid == 0:
        o_i[a, c] += result      // only lane 0 writes
```

The key insight: even though we can't parallelize over the E dimension (because $E > \text{SIMD width}$), we **can** parallelize the reduction over $Bc = 32$, which matches SIMD width exactly.

Memory Hierarchy

Global Memory (HBM)
 $Q[N, L, E], K[N_{kv}, S, E], V[N_{kv}, S, E]$

↓ load once per Q block

Threadgroup Memory (SRAM, 32KB)
 $q_local[Br][E] \leftarrow Q \text{ block, reused for all } Tc \text{ iterations}$
 $o_i[Br][E] \leftarrow \text{accumulated output}$

↓

Registers (per thread)
 m_i, l_i, s_a_b, p_a_b

K and V blocks are streamed from global memory in the inner loop over Tc . The Q block is loaded once into threadgroup memory and reused across all K/V tiles.

Implementation

In `flash_attention.metal`, write `flash_attention_f32_e128` with one threadgroup per (n, i) tile, where n is the flattened head batch and i is the query tile index. Use threadgroup memory for local Q and partial O , and use SIMD reductions (`simd_max`, `simd_sum`) for row-wise max/sum updates.

In `eval_gpu(...)`, load the kernel from the extension, bind inputs/outputs and scalar constants (`N`, `L`, `S`, `E`, head counts, `scale`, tile sizes), and dispatch over `(N, Tr, 1)`. Keep the same contiguous checks as CPU path. Also remember to add `src/flash_attention.metal` into `mlx_build_metallib(...)` in `CMakeLists.txt`.

You can test your implementation by running:

```
pdm run build-ext
pdm run test --week 2 --day 4 -- -k task_3
```

Task 4: Model Integration

```
src/tiny_llm/qwen3_week2.py
```

Finally, wire the kernel into model execution. Keep the existing grouped attention path as fallback, add the `use_flash_attention` switch in `Qwen3MultiHeadAttention`, and propagate `enable_flash_attn` from model initialization into each block. After KV cache update, build the correct causal mask for `L x S`, run attention in float32, and cast back to activation dtype.

You can run generation with Flash Attention enabled:

```
pdm run main --solution tiny_llm --loader week2 --model qwen3-0.6b --enable-flash-attn
```

You can also benchmark throughput with and without Flash Attention:

```
pdm bench --solution tiny_llm --loader week2 --model qwen3-0.6b
pdm bench --solution tiny_llm --loader week2 --model qwen3-0.6b --enable-flash-attn
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 2 Day 6 and 7: Chunked Prefill and Continuous Batching

In this chapter, we will implement **continuous batching**. The idea is to batch multiple requests together so we can make full use of the compute resources.

So far, we have assumed that the model only processes a single batch each time it is called. However, a single batch is usually not enough to saturate the compute resources. To address this, we can process multiple requests at the same time.

The first question is how to batch requests. A naive approach would be to select a fixed number of prompts (for example, 5) from the request queue and perform decoding as before. The problem is that different prompts produce sequences of different lengths. It is possible that 4 out of 5 requests finish decoding quickly, while the remaining one takes much longer. This leads to wasted compute resources and stalls all other requests.

A smarter approach is **continuous batching**. That is, we set the maximum number of requests we can process at once. When one request finishes, we replace its slot (i.e., its KV cache) with another request. In this way, the pipeline remains fully utilized.

Another challenge is how to handle decoding and prefilling at the same time. In this chapter, we adopt a simplified approach: we prefill one request, then decode one token for each request in progress. The general idea can be described with the following pseudocode:

```
while requests_in_queue_or_in_progress:
    if prefill_request exists:
        prefill_request.try_prefill() # perform a chunk of chunked prefill
    if prefill_request.ready:
        if kv_cache.try_add(prefill_request):
            prefill_request = next(requests)
    tokens = decode(model, kv_cache)
    requests.append(tokens)
```

We will also implement **chunked prefill** in this chapter. Prefilling a long prompt can take a significant amount of time. Since we are interleaving prefills and decodes, we want to reduce the latency of producing the next token. Ideally, the time slots for prefill and decode should be roughly equal. To achieve this, we can prefill a portion of the request at a time, using multiple slots to finish the entire prefill.

For prefilling, this essentially means providing a chunk of tokens to the model to populate the KV cache. For example:

```
# assume prompt_tokens is a list of 400 tokens and prefill chunk size is 128
_step(model, prompt_tokens[0:128], offset=0, kv_cache)
_step(model, prompt_tokens[128:256], offset=128, kv_cache)
_step(model, prompt_tokens[256:384], offset=256, kv_cache)
_step(model, prompt_tokens[384:400], offset=384, kv_cache)
```

Note that the causal mask generated during prefilling has the shape $L \times S$. For example, assume we already have 5 tokens in the KV cache and want to prefill 3 tokens. The mask should look like this:

0	0	0	-inf	-inf
0	0	0	0	-inf
0	0	0	0	0

This is the same masking logic you implemented in Week 1.

Task 1: Batch RoPE and Causal Mask for Prefill

```
src/tiny_llm/positional_encoding.py
src/tiny_llm/attention.py::causal_mask
```

Ensure your RoPE implementation accepts a `list[slice]` of offsets (one slice for sequence in the batch). Also, make sure your mask implementation correctly handles the case where $L \neq S$.

You can verify multi-offset RoPE, and that masking works for attention and flash attention with:

```
pdm run test --week 2 --day 6 -- -k task_1
```

Task 2: Batch KV Cache

```
src/tiny_llm/kv_cache.py::BatchingKvCache
```

The batch KV cache is a collection of KV caches, one for each request. A challenge here is generating a $B \times H \times L \times S$ mask for the batch, since requests can have different lengths.


```

S = max(S_i of the batch)
L = mask_length (input parameter)
keys: 1, H, S_i, D
values: 1, H, S_i, D
batched_keys: B, H, S, D
batched_values: B, H, S, D
mask: B, 1, L, S

```

You should fill the `batched_keys` and `batched_values` arrays so that each request's data is aligned at the end:

```

batched_keys[i, :, (S-S_i):S, :] = keys[i, :, :, :]
batched_values[i, :, (S-S_i):S, :] = values[i, :, :, :]
mask[i, :, 0:L, (S-S_i):S] = causal_mask(L, S_i)

```

You can verify your implementation by running:

```
pdm run test --week 2 --day 6 -- -k task_2
```

Task 3: Handle Batches in the Model

```
src/tiny_llm/qwen3_week2.py
```

Ensure your model can handle multiple requests simultaneously. You should also use the masks returned by the batch KV cache.

You should pass all of the tests by running:

```
pdm run test --week 2 --day 6 -- -k task_3
```

Task 4: Batch Generate

```
src/tiny_llm/batch.py
```

Implement `try_prefill` so that it prefills an entire request at once. Then implement the rest of the code as described in the starter code.

Task 5: Chunked Prefill

```
src/tiny_llm/batch.py
```

Modify `try_prefill` so that it performs prefilling in chunks, rather than all at once.

Note that you should materialize KV cache between chunks. Because MLX uses lazy evaluation, and chunked prefill keeps extending the KV cache across multiple model calls. If you never call `mx.eval`, the cache becomes a longer and longer lazy expression, so memory usage keeps growing. Calling `mx.eval` on the key and value tensors after each chunk materializes the current KV cache and truncates the graph.

You can test your implementation by running:

```
pdm run batch-main
```

This will use the `qwen3-0.6b` model with a batch size of 5 to process a fixed set of prompts.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 3 Day 1: Paged Attention, Part 1

In this chapter, we will design the **paged KV cache**. This is the storage abstraction behind paged attention.

By the end of Week 2, our serving stack already supports:

- per-request KV cache
- chunked prefill
- continuous batching
- FlashAttention

That gives us a working miniature serving engine, but the memory layout is still too simple. KV for each request is treated as one growing dense tensor, and batching rebuilds dense K/V for all active requests. That approach is easy to teach, but it does not scale well once requests become long and numerous.

Paged attention starts by fixing the storage layout.

Readings

- [vLLM Paged Attention Design](#)
- [Efficient Memory Management for Large Language Model Serving with PagedAttention](#)

Why the Week 2 KV Layout Becomes Expensive

Right now, the mental model looks like this:

```
request A -> one dense KV tensor
request B -> one dense KV tensor
request C -> one dense KV tensor
```

Before attention, the runtime repacks them into:

```
keys:    [B, H, S_max, D]
values:  [B, H, S_max, D]
mask:    [B, 1, L, S_max]
```

The trouble is that decode only adds a tiny amount of new information each step, but the dense layout keeps revisiting old KV.

For example, if a request already has 17 cached tokens and we decode 1 more token:

```
new useful work: append 1 token
dense repack view: rebuild 18 logical positions
```

For one request this is fine. For many live requests, the runtime spends more and more time moving previously computed KV instead of doing actual model work.

The Page Abstraction

Instead of storing each layer's KV for a request as one long tensor, we divide storage into fixed-size **pages**:

```
key_pages:  pages with up to page_size token slots
value_pages: pages with up to page_size token slots
```

Each layer cache keeps a small page table:

```
page_ids = [12, 5, 3]
context_len = 10
```

That means:

```
page 12 -> tokens 0..3
page  5 -> tokens 4..7
page  3 -> tokens 8..9
```

The logical sequence is still length 10. The difference is that the runtime is no longer forced to represent it as one contiguous tensor.

In our Day 1 teaching implementation, those fixed-size pages live in one shared **page pool** owned by the model. Every layer cache receives that same pool, but each layer cache keeps its own `page_ids`, `page_lens`, and `offset`.

In the reference solution, `page_size` is the physical page capacity. Unused tail slots are not part of the logical sequence; `page_lens` decides which prefix of each page is valid.

Why Fixed-Size Pages Help

The page abstraction gives us two immediate wins:

1. Appending a token usually updates only the current tail page in the pool.
2. Finished requests can return their pages to a shared free list.

This is the key memory-management idea behind paged attention systems such as vLLM.

Data Structures We Need

1. PagePool

The model should own one pool with a model-wide page allocator and flat K/V page storage:

```
free_pages: available page ids for the whole model
keys[page_id]: physical key page
values[page_id]: physical value page
```

Each layer still has distinct K/V contents because each layer cache allocates its own physical pages. In this teaching version, each layer cache also has its own logical page table. That is simpler than nano-vllm's shared block table: layer 0 might own pages `[0, 1]`, while layer 1 owns pages `[2, 3]`, but both page sets came from the same model-owned pool.

In the reference solution, this becomes `TinyKvPagedPool`.

2. PagedRequestCache

A layer cache for one request should track:

- `page_ids`
- `page_lens`
- `offset`
- `page_size`

Derived values:

- `num_pages = len(page_ids)`
- `context_len = offset`
- `last_page_fill = page_lens[-1]` when at least one page exists

In the reference solution, this becomes `TinyKvPagedCache`. It is created with a pool from the model. It should not allocate its own pool, because that would isolate one request from the shared page allocator.

The reference solution creates one `TinyKvPagedCache` per transformer layer. Those caches share the pool, but they do not share metadata: each layer cache owns its own `page_ids`, `page_lens`, and `offset`.

3. Tail-Append Logic

When new K/V arrives for one layer:

1. look at that layer cache's last page
2. if there is room, append only the new slice into the tail page
3. otherwise allocate a new page and continue writing
4. update cache metadata such as `page_lens` and `offset`

This replaces the Week 2 pattern of repeatedly concatenating along the sequence dimension.

Prefill with Pages

Suppose `page_size = 4` and one prefill chunk contains 6 tokens:

```
chunk = [t0 t1 t2 t3 t4 t5]
```

One possible layout is:

```
page 7 <- [t0 t1 t2 t3]
page 2 <- [t4 t5]          # 2 valid tokens, 2 unused slots of capacity
```

That layer cache's metadata becomes:

```
page_ids = [7, 2]
context_len = 6
```

The important property is that a later decode token can be appended to `page 2` without touching `page 7`.

Decode with Pages

During decode, each live request adds one token at a time.

With paged storage:

1. compute one-token `k` and `v`
2. check whether the tail page still has space
3. write into that page if possible
4. allocate a new page only when the old one is full

So if `page_size = 4` and `context_len = 9`:

```
page_ids = [12, 5, 3]
```

Appending token 9 only updates the last page instead of rebuilding all earlier KV.

Stage A: Keep Dense Attention

The cleanest first implementation is **paged storage with dense gather**.

That means:

- pages in the shared pool are the source of truth,
- layer caches stop owning one monolithic K/V tensor,
- layer caches only track page metadata,
- attention still receives dense K/V reconstructed from pages.

This is not the final paged attention runtime yet, but it is a very useful intermediate step:

- small surface-area change
- easier debugging
- direct correctness comparison against `TinyKvFullCache`

How This Maps to `tiny-llm`

`src/tiny_llm/paged_kv_cache.py`

Add:

- `TinyKvPagedPool`
- `TinyKvPagedCache`

Keep `TinyKvFullCache` in `src/tiny_llm/kv_cache.py` as a baseline and test oracle.

The key Day 1 behavior is:

1. write new K/V into the layer cache's tail page or newly allocated pages,
2. gather the layer cache's pages back into dense K/V,
3. feed that dense K/V into the old attention path.

So Day 1 changes the storage model first, not the attention kernel yet.

src/tiny_llm/batch.py

Requests should own per-layer cache handles instead of long dense K/V tensors.

The scheduler should still:

- perform chunked prefill,
- hold active requests,
- free cache pages when a slot finishes.

The difference is that freeing a request now means releasing all pages owned by its layer caches back to the pool.

Day 1 also keeps a small `rewind(n)` lifecycle hook. Rewind is useful for speculative decoding: if some drafted tokens are rejected, the cache must forget their K/V. In the paged cache, rewind frees whole pages that are no longer needed and shortens the valid length of the final remaining page.

Design Questions for Day 1

Before implementing, make sure the following are clear:

1. What page size should this repo use for teaching?
2. How do we represent the free-page allocator?
3. How do we prove that paged storage reconstructs the same logical KV as `TinyKvFullCache`?
4. How do layer cache handles share one pool while keeping their own page metadata?
5. When do we materialize page writes to avoid MLX lazy-graph growth?

Task 1: Design PagePool

src/tiny_llm/paged_kv_cache.py

Design a model-owned page pool that:

- owns the model-wide free-page allocator,
- stores flat fixed-size K/V pages,
- allocates and frees page ids,
- supports writing a chunk into page storage,
- is shared by every layer cache created by the model.

Task 2: Design PagedRequestCache

`src/tiny_llm/paged_kv_cache.py`

Replace the “one layer cache = one dense KV tensor” model with:

- `page_ids`
- `context_len`
- append logic over fixed-size pages
- `release()` for returning pages on request completion
- `rewind(n)` for dropping the newest `n` logical tokens

Task 3: Add a Dense-Gather Compatibility Path

`src/tiny_llm/paged_kv_cache.py`
`src/tiny_llm/qwen3_week3.py`

Build a compatibility path that reconstructs dense K/V from pages and compares it against `TinyKvFullCache`.

This gives us a correctness check before we change the attention path itself.

In the next chapter, we will take the next step: instead of gathering dense K/V before attention, we will pass runtime metadata such as `block_table` directly into a paged attention path.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 3 Day 2: Paged Attention, Part 2

In this chapter, we move from **paged KV storage** to the runtime metadata and execution path needed for **real paged attention**.

Part 1 introduced fixed-size pages, a model-owned page pool shared by layer caches, and per-layer page metadata. That change already improves the storage abstraction, but it does not yet remove the dense gather before attention. To get the full benefit, the attention path itself must understand how to read from pages.

Paged KV Cache vs Paged Attention

These two ideas are related, but they are not the same:

1. **Paged KV cache** KV is stored in fixed-size pages.
2. **Paged attention** The attention path reads KV directly from those pages via metadata such as a page table.

You can implement the first one without the second one, but the real serving payoff comes when both are present.

The Metadata a Paged Runtime Needs

Once KV is paged, dense $B \times H \times S \times D$ tensors are no longer the natural runtime representation. Instead, the runtime should prepare metadata like:

```
block_table: [B, max_pages_per_request]
context_lens: [B]
```

For the current layer being executed:

- `block_table[b, i]` gives the page id for request `b`'s current-layer logical page `i`
- `context_lens[b]` gives the valid token count for request `b`

This is the bridge between the scheduler and the attention kernel.

A production runtime often also carries write-side metadata such as `slot_mapping`. For this chapter, we keep the write side inside the cache and focus on the read-side metadata needed by attention.

Why `block_table` Matters

Suppose one layer cache for request A has:

```
page_ids = [12, 5, 3]
context_len = 10
page_size = 4
```

Then the logical sequence positions map to physical storage like this:

```
logical 0..3 -> page 12
logical 4..7 -> page 5
logical 8..9 -> page 3
```

The attention runtime does not need a fully gathered dense tensor if it already knows:

- which current-layer page each logical block lives in,
- how long the context is,
- and where the current query positions are.

That is exactly what `block_table` and `context_lens` encode.

The Real Attention API

At this point, the runtime should grow a new attention entry point:

```
paged_attention(
    query,
    key_pages,
    value_pages,
    block_table,
    context_lens,
    page_size,
    scale=None,
    mask="causal",
)
```

With shapes like:

```
query:          B, H_q, L, D
key_pages[i]:   1, H_kv, page_size, D
value_pages[i]: 1, H_kv, page_size, D
block_table:    B, max_pages
context_lens:   B
```

Compared with the Week 2 dense path, the important difference is that the source length is no longer represented as one contiguous tensor dimension. It is reconstructed logically

from the page table.

In this chapter, `paged_attention` should read pages directly from a GPU kernel. The runtime contract is now: model code and batching code pass pages plus metadata, and the attention kernel walks that metadata without first rebuilding dense K/V.

Prefill Metadata

During prefill, a chunk may span multiple pages. The runtime needs to know:

- which current-layer pages already existed,
- which new pages were allocated,
- how many valid tokens are in the tail page,
- how to map incoming K/V rows into page storage.

In this teaching implementation, the cache still owns the write-side bookkeeping. The attention path only needs the block table after the write is done.

Decode Metadata

During decode, each active request typically writes one token.

The runtime should be able to:

1. append the token's K/V to the current tail page,
2. allocate a new page only if the tail page is full,
3. update the current layer cache's `context_len`,
4. run attention over the full logical context using `block_table`

This is the point where decode stops paying the repeated dense-repack cost from Week 2.

How This Maps to `tiny-llm`

`src/tiny_llm/attention.py`

Add a new function:

```
def paged_attention(...):  
    ...
```

For this course implementation, make it a FlashAttention-style page-walking Metal kernel:

1. use `block_table[b]` to find the physical pages for request `b`,
2. use `context_lens[b]` to ignore unused tail capacity,
3. visit K/V in small tiles instead of materializing dense K/V,
4. merge each tile into the output with online softmax.

The important change from Week 2 is the K/V address calculation. Week 2 can advance through dense K/V by pointer arithmetic. Week 3 must translate each logical key position through `block_table` first:

`logical key position -> logical page -> physical page id -> slot in page`

After that lookup, the online-softmax update is the same idea as Week 2 FlashAttention. We still avoid a dense K/V gather before attention.

The page pool should therefore expose contiguous physical storage:

```
key_pages:  P, H_kv, page_size, D
value_pages: P, H_kv, page_size, D
```

A Python list of page tensors is convenient for teaching the allocator, but a GPU kernel needs a single buffer so `page_id` can be turned into an address.

src/tiny_llm/qwen3_week3.py

The attention module should call the paged runtime directly:

```
metadata = cache.update_and_fetch_paged(...)
x = paged_attention(...)
```

Week 3 cache handles are expected to provide paged metadata. If a dense cache is passed to the Week 3 model, that is a programming error rather than a signal to silently fall back to Week 2 attention.

src/tiny_llm/batch.py

The scheduler now needs to prepare runtime metadata instead of only dense K/V:

- per-layer page tables for each active request
- padded batch `block_table`
- `context_lens`

This is where continuous batching and paged attention finally connect. In Week 2, batching worked by repacking tensors. In Week 3, batching should work by reusing page tables and

updating only the new slots.

Recommended Incremental Rollout

The safest implementation order is:

1. paged storage
2. `block_table` / `context_lens` plumbing
3. FlashAttention-style page-walking GPU attention
4. model and batch dispatch

This order matters because it gives us a clean correctness baseline at each step.

Correctness Invariants

These are the invariants worth checking in tests:

1. `context_len` always equals the number of written logical token positions.
2. `block_table` reconstructs the same logical KV order as the dense baseline.
3. the allocator never hands the same page to two live cache handles unless explicit sharing is implemented.
4. releasing a request returns all pages owned by all of its layer caches exactly once.
5. decode allocates a new page only when the tail page overflows.

Task 1: Add Batch Metadata

```
src/tiny_llm/paged_kv_cache.py
src/tiny_llm/kv_cache.py
src/tiny_llm/batch.py
```

Extend the batch cache and scheduler so they can prepare:

- `block_table`
- `context_lens`

for all active requests.

Task 2: Define paged_attention

```
src/tiny_llm/attention.py  
src/extensions/src/paged_attention.cpp  
src/extensions/src/paged_attention.metal
```

Add a paged attention interface whose inputs come from the paged runtime rather than a dense reconstructed `s` dimension.

The reference solution walks every request's block table and keeps online softmax state:

```
running_max = max(previous_max, page_max)  
running_sum = previous_sum * exp(previous_max - running_max) + page_sum  
output = previous_output * exp(previous_max - running_max) + page_output
```

After all visible pages are consumed, divide `output` by `running_sum`. This is the key idea that lets the kernel avoid materializing dense K/V while still producing the same result as dense attention.

Task 3: Dispatch from the Model

```
src/tiny_llm/qwen3_week3.py
```

Update the model so it can route to paged attention when the cache provides paged runtime metadata.

Task 4: Connect It to Continuous Batching

```
src/tiny_llm/batch.py
```

Update request admission, slot reuse, and request removal so that:

- finished requests free their pages,
- in this teaching implementation, that means freeing pages from every layer cache,
- new requests allocate from the shared pool,
- active decode steps reuse page metadata instead of rebuilding dense K/V.

After this chapter, the serving stack has the right structure for a real high-throughput runtime: paging is no longer just a storage trick, but part of the execution model itself.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Week 3 Day 3: Mixture of Experts

In this chapter, we will implement the feed-forward shape of **Mixture of Experts**, or **MoE**, for the Qwen3 family.

So far, every transformer block in tiny-llm has used the same dense Qwen3 MLP:

```
x -> gate_proj
x -> up_proj
SiLU(gate_proj(x)) * up_proj(x) -> down_proj
```

That is a SwiGLU MLP. Every token visits the same weights.

MoE changes only the feed-forward half of the transformer block. Instead of one dense MLP, the model owns many expert MLPs. A small router chooses which experts each token should use:

```
token hidden state -> router -> top-k experts -> weighted expert outputs
```

The attention path does not change. KV cache does not change. The sparse work is inside the MLP half of the block.

Readings

- [Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer](#)
- [GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding](#)
- [Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity](#)

Dense MLP vs MoE MLP

The dense Qwen3 MLP from Week 1 has one set of weights:

```
w_gate: hidden_dim, dim
w_up:   hidden_dim, dim
w_down: dim, hidden_dim
```

A Qwen3-MoE sparse block has a bank of those weights:

```
expert_gate: num_experts, moe_hidden_dim, dim
expert_up:   num_experts, moe_hidden_dim, dim
expert_down: num_experts, dim, moe_hidden_dim
```

The router produces one score per expert:

```
router_logits: B, L, num_experts
router_probs:  softmax(router_logits)
```

Then the model picks `num_experts_per_tok` experts for each token:

```
expert_ids:      B, L, num_experts_per_tok
expert_scores: B, L, num_experts_per_tok
```

For each token, only those selected experts run. Their outputs are weighted and summed:

```
output[token] = sum(score_i * expert_i(token))
```

That is the central MoE idea: the model can contain many parameters, but each token activates only a small subset of them.

Qwen3-MoE Shape

Qwen3-MoE keeps the same attention structure as Qwen3, including QK norm, GQA, RoPE, and the same KV cache interface. It replaces some dense MLP layers with a sparse MoE block.

The useful pieces are:

- `gate` : a router linear layer from hidden size to `num_experts`
- `switch_mlp` : many SwiGLU experts with `moe_intermediate_size`
- `num_experts_per_tok` : how many experts a token uses
- `norm_topk_prob` : whether selected expert scores are renormalized
- `decoder_sparse_step` and `mlp_only_layers` : which layers are sparse vs dense

There is no shared expert in the Qwen3-MoE block we are following. The sparse feed-forward output is just the weighted top-k expert mixture.

Grouped Quantized Matmul

MLX does not give us a single high-level MoE block in `mlx.nn`. It does have a lower-level primitive, `mx.gather_qmm`, that performs quantized matrix multiplication while selecting a different matrix for each row. In this chapter, we will build a narrow teaching version of that idea: `grouped_quantized_matmul`.

For MoE, that means:

```

token rows:  N, D
expert ids:   N
weights:     E, 0, D packed as 4-bit QuantizedWeights
output:      N, 0

```

The row with `expert_ids[i] = e` should multiply by `weights[e]` .

Task 1 will assume the rows are already sorted by expert id. The MoE helper will keep the inverse order from the sort so the result can be restored to the original token order.

Router Step

The router is just a quantized linear layer:

```

router_logits = quantized_linear(x, w_router)
router_probs = softmax(router_logits, axis=-1)

```

For a batch of tokens:

```

x:           B, L, D
router_logits: B, L, E
router_probs:  B, L, E

```

where `E = num_experts` .

Qwen3-MoE then uses top-k selection:

```

expert_ids = argpartition(-router_probs, k)[:k]
expert_scores = take_along_axis(router_probs, expert_ids)

```

If `norm_topk_prob` is true, renormalize `expert_scores` so the selected scores sum to 1 for each token.

Expert Step

Each expert is the same kind of SwiGLU MLP we already know:

```

expert(x) = down_proj(SiLU(gate_proj(x)) * up_proj(x))

```

The implementation should build token-expert jobs, group them by expert, and run the expert projections with `grouped_quantized_matmul` :

```

selected expert ids -> expanded token-expert rows
expanded rows -> sort/group by expert id
grouped expert rows -> grouped gate/up projection
SiLU(gate) * up -> grouped down projection
restore original token/top-k order -> weighted sum

```

The reorder is part of the model implementation. It keeps all token rows for the same expert contiguous so the expert bank can be applied with grouped matrix multiplication.

Task 1: Grouped Quantized Matmul

```

src/extensions/src/quantized_matmul.cpp
src/extensions/src/quantized_matmul.metal
src/tiny_llm/quantize.py
src/tiny_llm/moe.py

```

Implement `grouped_quantized_matmul`, then use it from `grouped_expert_linear`. This is the quantized grouped-matmul core of MoE.

`grouped_quantized_matmul` accepts:

```

a:          R, D
w_experts:  packed QuantizedWeights for num_experts, output_dim, D
expert_ids:  R, sorted by expert id

```

It returns:

```

out:          R, output_dim

```

Each row uses the expert selected by the matching row in `expert_ids`:

```

out[row] = a[row] @ dequantize(w_experts[expert_ids[row]]).T

```

The implementation should:

1. add a Python wrapper for `grouped_quantized_matmul`,
2. extend the quantized matmul extension with a grouped entrypoint,
3. read `expert_ids[row]` inside the kernel,
4. use that expert id to choose the expert weight, scale, and bias row.

After that, implement `grouped_expert_linear` in `src/tiny_llm/moe.py`:

1. flatten token rows and expert ids,
2. sort rows by expert id,
3. call `grouped_quantized_matmul`,
4. restore the original order.

The call should look like:

```
out = grouped_quantized_matmul(
    w_experts.scales,
    w_experts.biases,
    group_size=w_experts.group_size,
    bits=w_experts.bits,
    a=grouped_rows,
    b=w_experts.weight,
    expert_ids=grouped_expert_ids,
    transpose_b=True,
)
```

This task maps to the same idea as `QuantizedSwitchLinear` in `mlx-lm`: each token row uses a different packed expert matrix, and the expert ids choose the right matrix.

Task 2: Router Top-k

`src/tiny_llm/moe.py`

Implement `route_topk`. It accepts hidden states and router weights, then returns:

- router probabilities
- selected expert ids
- selected expert scores

Use `quantized_linear` and `softmax`. Use `mx.argmaxpartition` to select the top `num_experts_per_tok` experts, then `mx.take_along_axis` to gather their scores.

Keep `norm_topk_prob` as an argument because Qwen3-MoE stores this behavior in the model config.

Task 3: Qwen3 Sparse MoE Block

`src/tiny_llm/moe.py`

Implement `Moe` by composing Task 1 and Task 2:

```

hidden states -> route_topk
hidden states + expert ids -> grouped gate projection
hidden states + expert ids -> grouped up projection
SiLU(gate) * up -> grouped down projection
weighted sum over num_experts_per_tok

```

This completes the Qwen3-MoE sparse feed-forward block. There is no shared expert branch in this block.

Task 4: Integrate Qwen3-MoE Layers

```

src/tiny_llm/qwen3_week3.py
src/tiny_llm/models.py

```

Add a Qwen3-MoE loader path that reuses the Week 3 Qwen3 attention and paged KV cache behavior, but swaps selected block MLPs for `Moe`.

The model wrapper should:

- keep Qwen3 attention unchanged,
- use regular `Qwen3MLP` for `mlp_only_layers`,
- use `Moe` for sparse layers selected by `decoder_sparse_step`,
- load router and expert weights as `QuantizedWeights` from the Qwen3-MoE MLX model,
- preserve the same decode call shape:

```
logits = model(tokens, offset, cache)
```

No scheduler API change in `src/tiny_llm/batch.py` is required for correctness.

Run this task through the normal generation entrypoints instead of adding a separate unit test. For example:

```
hf download Qwen/Qwen3-30B-A3B-MLX-4bit
```

```
pdm run main --solution tiny_llm --loader week3 --model qwen3-30b-a3b \
  --prompt "Give me a short introduction to mixture of experts."
```

```
pdm run batch-main --solution tiny_llm --loader week3 --model qwen3-30b-a3b \
  --batch-size 2 --prefill-step 16
```

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.

Glossary Index

- [Scaled Dot Product Attention](#)
- [Multi Head Attention](#)
- [Linear](#)
- [Rotary Positional Encoding](#)
- [Grouped Query Attention](#)
- [Qwen3 Attention Module](#)
- [RMSNorm](#)
- [SiLU](#)
- [SwiGLU](#)
- [MLP](#)
- [Embedding](#)
- [Qwen3 Transformer Block](#)
- [Week 1 Qwen3 Model](#)
- [dequantize_linear](#)
- [KV Cache](#)
- [Quantized Matmul](#)
- [Flash Attention](#)
- [Paged Attention](#)

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/tiny-llm.

tiny-llm-book © 2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.